

STAR Robot Control System

Software Architecture Primer

Comprehensive Technical Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1	Glossary of Key Terms	4
1.1	Communication Protocols	4
1.2	Middleware and Infrastructure	4
1.3	Hardware Interfaces	5
1.4	Software Concepts	6
1.5	Protocols and Formats	6
1.6	Deployment and Operations	7
2	Executive Summary	8
2.1	Architecture Decisions Summary	8
3	Communication Protocol Analysis	9
3.1	The Problem Statement	9
3.2	Protocol Options Evaluated	9
3.3	Detailed Comparison	9
3.4	Protocol Deep Dive	9
3.4.1	gRPC + GraphQL Hybrid (Selected)	10
3.4.2	Why This Was Selected for STAR	10
3.5	Packet Format Comparison	11
3.5.1	Protobuf (gRPC) Binary Encoding	11
3.5.2	JSON (GraphQL) Text Encoding	11
3.5.3	Serialization Performance	11
4	System Architecture	12
4.1	Network Topology	12
4.2	Dual WiFi Mode Operation	12
4.3	Software Stack Overview	13
5	SLAM Algorithm Selection	14
5.1	Algorithm Comparison	14
5.2	Selection: SLAM Toolbox	14
5.3	SLAM Toolbox Configuration	14
5.4	Why Not Other Algorithms?	15
5.5	Nav2 DWA Controller Parameters	15
6	ML Inference Pipeline	18
6.1	Model Selection	18
6.2	Model Architecture	18
6.3	Inference Pipeline	18
6.4	Performance Optimization	20
6.5	Integration with Safety System	20
7	CV Safety System	22
7.1	Safety Zones	22
7.2	Distance Estimation	23
7.3	Safety State Machine	24
7.4	Integration with Navigation	24

8	Battery Management	26
8.1	Battery State Thresholds	26
8.2	Battery Monitoring	26
8.3	Safe Shutdown Sequence	28
9	Repository Structure	29
9.1	Directory Layout	29
10	Spring Boot Gateway Implementation	31
10.1	Build Configuration	31
10.2	Application Configuration	32
10.3	Six-Layer Software Architecture	34
10.3.1	Layer Responsibilities	34
10.3.2	Control Timing Hierarchy	35
10.4	GraphQL Schema	36
10.5	gRPC Protobuf Definition	38
10.6	Service Implementations	39
10.6.1	Motor Control Service	39
10.6.2	ESP32 SPI Adapter	42
10.6.3	ESP32 TCP Adapter	45
10.6.4	GraphQL Controller	47
10.6.5	gRPC Service	49
11	PWA Frontend Implementation	51
11.1	Package Configuration	51
11.2	Vite PWA Configuration	52
11.3	Apollo Client with Offline Support (Vue 3)	54
11.4	GraphQL Operations	56
11.5	gRPC-Web Client	60
11.6	Robot Connection Composable (Vue 3 - Replaces useMockData)	62
11.7	IndexedDB Offline Storage	66
12	ESP32 Firmware Updates	71
12.1	TCP Server Module	71
12.2	Main Application Integration	77
13	Process Flow Diagrams	78
13.1	Motor Command Flow	78
13.2	SLAM Data Flow	78
13.3	Navigation Pipeline	78
13.4	Telemetry Aggregation Flow	79
14	Deployment Configuration	80
14.1	Systemd Service	80
14.2	Avahi mDNS Service	81
14.3	Envoy Proxy Configuration	81
15	Implementation Checklist	84
15.1	Phase 1: Infrastructure Setup	84

15.2 Phase 2: Backend Core	84
15.3 Phase 3: Frontend PWA	84
15.4 Phase 4: ESP32 Updates	85
15.5 Phase 5: Deployment	85
15.6 Phase 6: Documentation	85
16 Technology Summary	86
17 References and Resources	87
17.1 Protocol Documentation	87
17.2 Framework Documentation	87
17.3 Research References	87
17.4 Build Tools	87

1 Glossary of Key Terms

This section defines key terminology used throughout this document.

1.1 Communication Protocols

gRPC **Google Remote Procedure Call** – A high-performance, open-source RPC framework developed by Google. Uses HTTP/2 for transport and Protocol Buffers (Protobuf) for serialization. Enables efficient binary communication between services with built-in support for streaming.

GraphQL

A query language for APIs developed by Facebook. Unlike REST, clients specify exactly what data they need, reducing over-fetching. Supports queries (read), mutations (write), and subscriptions (real-time updates via WebSocket).

gRPC-Web

A JavaScript implementation of gRPC for browser clients. Since browsers cannot use HTTP/2 trailers directly, gRPC-Web requires a proxy (like Envoy) to translate between gRPC-Web and native gRPC.

Protobuf **Protocol Buffers** – Google’s language-neutral, platform-neutral binary serialization format. Smaller and faster than JSON. Messages are defined in `.proto` files and compiled to language-specific code.

WebSocket

A protocol providing full-duplex communication channels over a single TCP connection. Used by GraphQL subscriptions for real-time data streaming from server to client.

REST **Representational State Transfer** – An architectural style for web APIs using HTTP methods (GET, POST, PUT, DELETE). Uses JSON for data exchange. Simpler but less efficient than gRPC for high-frequency communication.

1.2 Middleware and Infrastructure

DDS **Data Distribution Service** – A middleware protocol and API standard for data-centric publish-subscribe communication. Used by ROS 2 for inter-node communication. Provides real-time, scalable, and reliable data exchange with Quality of Service (QoS) policies.

ROS 2 **Robot Operating System 2** – An open-source robotics middleware framework. Provides tools, libraries, and conventions for building robot applications. Uses DDS for communication between nodes.

SLAM **Simultaneous Localization and Mapping** – A computational technique where a robot builds a map of an unknown environment while simultaneously tracking its location within that map.

Nav2 **Navigation 2** – The ROS 2 navigation stack. Provides autonomous navigation capabilities including path planning (A*), obstacle avoidance (DWA), and localization (AMCL).

DWA	Dynamic Window Approach – A local path planning algorithm used by Nav2. Samples velocities in a “dynamic window” (velocities reachable within one control cycle) and scores trajectories based on proximity to goal, obstacle clearance, and velocity. Runs at 20 Hz in the STAR robot.
AMCL	Adaptive Monte Carlo Localization – A probabilistic localization algorithm that uses a particle filter to track the robot’s pose within a known map. Particles represent hypotheses about the robot’s position; sensor observations update particle weights. SLAM Toolbox provides similar functionality during mapping.
mDNS	Multicast DNS – A protocol that resolves hostnames to IP addresses within small networks without a local DNS server. Enables <code>star-robot.local</code> hostname resolution. Implemented by Avahi on Linux.
Avahi	A free, open-source implementation of mDNS/DNS-SD (Service Discovery) for Linux. Allows devices to discover each other on a local network without configuration.
Envoy	A high-performance, open-source edge and service proxy. Used in this project to translate gRPC-Web requests from browsers to native gRPC for the Spring Boot backend.

1.3 Hardware Interfaces

SPI	Serial Peripheral Interface – A synchronous serial communication interface for short-distance communication. Uses 4 wires: COPI (Controller Out Peripheral In), CPIO (Controller In Peripheral Out), CLK (Clock), and CS (Chip Select). Used for RPi5-to-ESP32 communication at 10 MHz.
I2C	Inter-Integrated Circuit – A two-wire serial communication protocol (SDA for data, SCL for clock). Used for connecting sensors like IMUs. Supports multiple devices on the same bus with unique addresses.
UART	Universal Asynchronous Receiver-Transmitter – A serial communication protocol using TX (transmit) and RX (receive) lines. Commonly used for debugging and some sensor connections.
GPIO	General Purpose Input/Output – Configurable digital pins on microcontrollers and single-board computers. Can be set as input (read sensors/buttons) or output (control LEDs/relays).
PWM	Pulse Width Modulation – A technique for controlling analog devices using digital signals. The duty cycle (percentage of time the signal is HIGH) controls the effective power. Used for motor speed control.
Center-Aligned PWM	A PWM mode where the counter counts up then down, centering pulses in the period. Produces cleaner signals with reduced electromagnetic interference (EMI) compared to edge-aligned PWM. Used by the ESP32 MCPWM peripheral for motor control.
PID	Proportional-Integral-Derivative – A control loop mechanism for maintaining a desired setpoint. The ESP32 runs a 1 kHz PID loop to maintain precise motor velocities based on encoder feedback.

- SMBus** **System Management Bus** – A two-wire interface based on I2C but with stricter timing requirements and additional features like packet error checking (PEC) and timeouts. Used for battery fuel gauge ICs and power management chips.
- 1-Wire** A serial protocol using a single data wire (plus ground) for communication. Devices are powered parasitically from the data line or via a separate power pin. Used by DS18B20 temperature sensors for thermal monitoring.

1.4 Software Concepts

- PWA** **Progressive Web App** – A web application that uses modern web technologies to deliver app-like experiences. Can work offline, be installed on devices, and receive push notifications.

Service Worker

A JavaScript worker that runs in the background, separate from the web page. Enables offline functionality by intercepting network requests and serving cached responses.

IndexedDB

A low-level browser API for storing large amounts of structured data. Used for offline telemetry storage in the PWA.

Apollo Client

A comprehensive GraphQL client for JavaScript. Provides caching, state management, and offline support for GraphQL operations.

- Workbox** A set of JavaScript libraries from Google for adding offline support to web apps. Simplifies service worker creation and caching strategies.

Coroutines

A Kotlin feature for asynchronous programming. Allows writing non-blocking code in a sequential style. Used extensively in the Spring Boot gateway for handling concurrent requests.

- Pi4J** A Java library for accessing GPIO, I2C, SPI, and other hardware interfaces on Raspberry Pi. Used by the gateway to communicate with the ESP32 via SPI.

FreeRTOS

A real-time operating system kernel for embedded devices. Used by the ESP32 firmware for task scheduling, ensuring the motor control loop runs at exactly 1 kHz.

1.5 Protocols and Formats

SLAMTEC Protocol

The communication protocol used by RPLiDAR sensors (like the RPLiDAR C1). A proprietary serial protocol over UART for commanding the sensor and receiving scan data, supported by the `rplidar_ros2` driver.

- JSON** **JavaScript Object Notation** – A lightweight, human-readable data interchange format. Used by GraphQL and REST APIs. Larger than binary formats like Protobuf.

HTTP/2 The second major version of HTTP. Supports multiplexing (multiple requests over one connection), header compression, and server push. Required by gRPC.

TCP/IP **Transmission Control Protocol / Internet Protocol** – The foundational communication protocols of the internet. Provides reliable, ordered delivery of data packets.

1.6 Deployment and Operations

systemd The init system and service manager for modern Linux distributions. Manages service lifecycle, auto-restart on failure, and dependency ordering.

Buildroot

A tool for building embedded Linux systems. Used to create the minimal custom Linux image for the Raspberry Pi 5.

Prometheus

An open-source monitoring and alerting toolkit. Collects metrics from applications via HTTP endpoints. Used with Spring Boot Actuator for gateway monitoring.

Actuator A Spring Boot module that exposes operational information (health, metrics, info) via HTTP endpoints. Integrates with Prometheus for metrics collection.

JVM **Java Virtual Machine** – The runtime environment for Java and Kotlin applications. The Spring Boot gateway runs on the JVM.

2 Executive Summary

This document provides a comprehensive technical reference for implementing the STAR robot control system software stack. It covers:

- **Protocol Selection:** gRPC + GraphQL hybrid architecture with detailed justification
- **Backend Gateway:** Kotlin/Spring Boot 3 implementation for RPi5
- **Frontend PWA:** Vue 3 + TypeScript Progressive Web App with full offline capability
- **Hardware Integration:** SPI and TCP communication with ESP32
- **Deployment:** Systemd services, mDNS discovery, and metrics

2.1 Architecture Decisions Summary

Decision	Choice	Rationale
Communication Protocol	gRPC + GraphQL Hybrid	Low latency + flexible queries
Backend Framework	Kotlin/Spring Boot 3	JVM performance + coroutines
Frontend Framework	Vue 3 + TypeScript + Vite	Type-safe, reactive UI with Pinia state
Offline Strategy	Workbox + Apollo Cache	Full offline with IndexedDB
ESP32 Communication	SPI (primary) + TCP (fallback)	Redundancy + flexibility
Service Discovery	Avahi/mDNS	Zero-config networking
Metrics	Prometheus + Actuator	Industry standard

Table 1: Key Architecture Decisions

3 Communication Protocol Analysis

3.1 The Problem Statement

The STAR robot requires communication across three distinct paths:

1. **Controller App** → **RPi5 Gateway**: Over WiFi/Internet (this is the focus)
2. **RPi5 Gateway** → **ESP32**: Over SPI bus (already defined)
3. **RPi5 Gateway** → **LiDAR**: Over UART (already defined)

The critical question is: *What protocol should Path #1 use?*

3.2 Protocol Options Evaluated

Four options were evaluated for controller-to-robot communication:

1. gRPC + GraphQL Hybrid
2. ConnectRPC (modern gRPC alternative)
3. WebSocket + GraphQL
4. Pure gRPC with Envoy Proxy

3.3 Detailed Comparison

Aspect	gRPC + GraphQL	ConnectRPC	WebSocket + GraphQL	Pure gRPC
Latency	gRPC: 1-5ms, GraphQL: 10-30ms	5-15ms	WS: 5-10ms, GQL: 10-30ms	1-5ms
Packet Size	Binary (gRPC) + JSON (GQL)	Binary OR JSON	JSON only	Binary only
Browser Support	gRPC-Web (limited) + GQL (full)	Full (HTTP/1.1)	Full native	Requires Envoy
Bidirectional	gRPC-Web: NO, GQL Sub: YES	NO	YES (native)	gRPC-Web: NO
Infrastructure	2 servers	1 server	1 server + WS	1 server + Envoy
Learning Curve	High (2 systems)	Medium	Medium	Medium-High
Ecosystem	Very mature (both)	Newer (2022+)	Very mature	Very mature

Table 2: Protocol Comparison Matrix

3.4 Protocol Deep Dive

3.4.1 gRPC + GraphQL Hybrid (Selected)

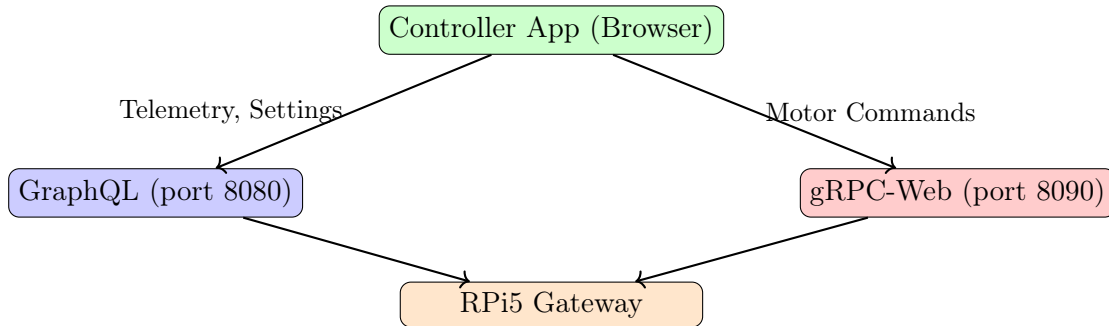


Figure 1: gRPC + GraphQL Hybrid Architecture

Advantages:

- gRPC binary format provides fastest motor command transmission
- GraphQL enables flexible queries for complex UI data needs
- Clear separation of concerns: real-time control vs. data queries
- Both protocols are industry standards with excellent tooling

Disadvantages:

- Two different APIs to maintain
- gRPC-Web requires Envoy proxy as translator
- More complex client code (two client libraries)

3.4.2 Why This Was Selected for STAR

1. **Low-Latency Motor Control:** gRPC's binary Protobuf format minimizes serialization overhead for time-critical motor commands.
2. **Flexible Telemetry Queries:** GraphQL subscriptions provide real-time telemetry streaming with selective field fetching.
3. **Offline Capability:** Apollo Client (GraphQL) has excellent caching and offline support built-in.
4. **Latency Acceptability:** For motor commands, even 10-30ms latency from browser to RPi5 is acceptable because:
 - The ESP32 maintains autonomous 1kHz PID control between commands
 - Human reaction time is approximately 200-300ms
 - The critical real-time path is RPi5 → ESP32 (SPI), not Controller → RPi5

3.5 Packet Format Comparison

3.5.1 Protobuf (gRPC) Binary Encoding

Protobuf uses a compact binary format that excludes field names:

```

1 message VelocityCommand {
2     float left_velocity = 1;    // 4 bytes + tag
3     float right_velocity = 2;   // 4 bytes + tag
4     uint64 timestamp = 3;       // 1-10 bytes (varint)
5 }
```

Listing 1: Protobuf Message Definition

Wire format example (VelocityCommand with left=1.0, right=0.5, timestamp=1702500000000):

```

0D 00 00 80 3F  // field 1 (float): 1.0
15 00 00 00 3F  // field 2 (float): 0.5
18 80 A0 CF EE E5 31  // field 3 (varint): timestamp
```

Total size: approximately 17 bytes

3.5.2 JSON (GraphQL) Text Encoding

Equivalent JSON payload:

```

1 {
2     "leftVelocity": 1.0,
3     "rightVelocity": 0.5,
4     "timestamp": 1702500000000
5 }
```

Listing 2: JSON Equivalent

Total size: approximately 65 bytes (3.8x larger)

3.5.3 Serialization Performance

Based on industry benchmarks¹:

Metric	Protobuf	JSON	Difference
Message Size	17 bytes	65 bytes	3.8x smaller
Serialization (JVM)	1.2 μ s	4.5 μ s	3.75x faster
Deserialization (JVM)	0.8 μ s	3.2 μ s	4x faster
Network Latency (1Mbps)	0.14 ms	0.52 ms	3.7x faster

Table 3: Protobuf vs JSON Performance Comparison

¹Source: Ambassador Labs, Protobuf vs JSON Performance Analysis, 2024

4 System Architecture

4.1 Network Topology

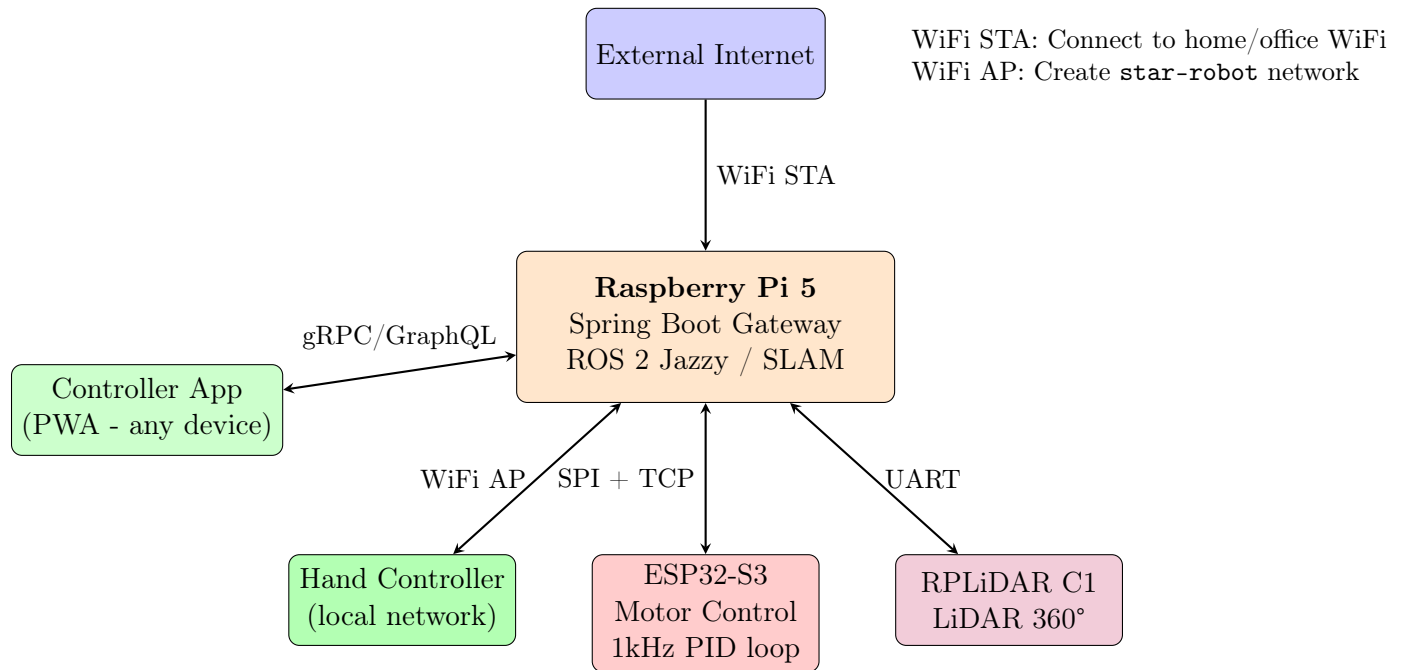


Figure 2: Complete Network Topology

4.2 Dual WiFi Mode Operation

The Raspberry Pi 5 operates in **simultaneous dual WiFi mode**:

1. **Station (STA) Mode:** Connects to external internet for remote access
 - Enables control from anywhere with internet access
 - Allows firmware updates and remote debugging
2. **Access Point (AP) Mode:** Creates local **star-robot** network
 - Provides low-latency local control
 - ESP32 connects to this network for TCP fallback
 - Hand controller connects for direct operation

4.3 Software Stack Overview

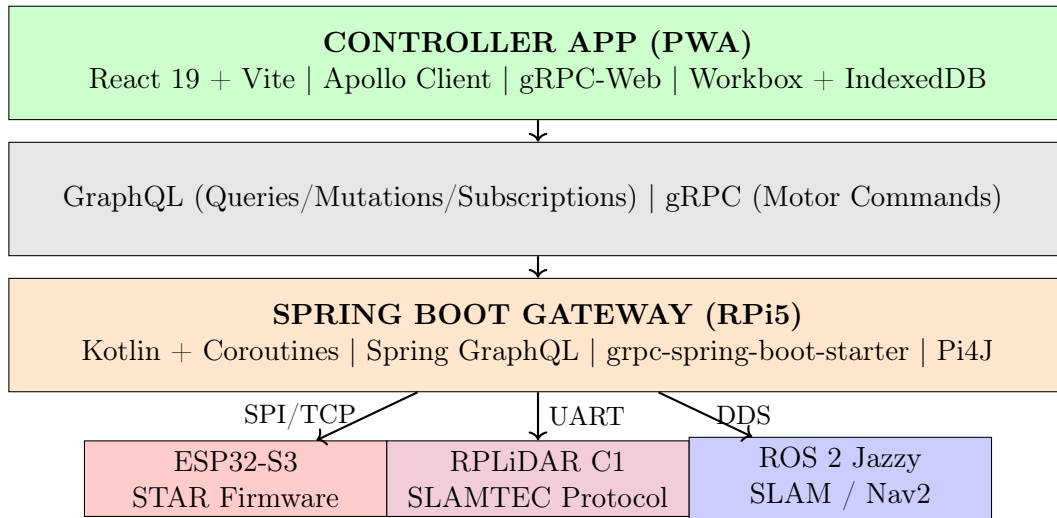


Figure 3: Software Stack Architecture

5 SLAM Algorithm Selection

The STAR robot uses 2D SLAM (Simultaneous Localization and Mapping) for indoor navigation. This section documents the algorithm selection process and rationale.

5.1 Algorithm Comparison

Four SLAM algorithms were evaluated for the STAR platform:

Table 4: SLAM Algorithm Comparison Matrix

Criterion (Weight)	SLAM Toolbox	Cartographer	Hector SLAM	GMapping
Compute Efficiency (0.25)	5	3	4	4
Loop Closure Quality (0.20)	4	5	2	3
Indoor Accuracy (0.25)	5	4	4	4
ROS 2 Support (0.15)	5	4	3	4
Memory Usage (0.15)	4	3	5	4
Weighted Score	4.55	3.75	3.55	3.80

Scoring: 1 = Poor, 2 = Below Average, 3 = Average, 4 = Good, 5 = Excellent

5.2 Selection: SLAM Toolbox

SLAM Toolbox was selected as the primary SLAM algorithm based on:

1. **Compute Efficiency** – Optimized for resource-constrained platforms like RPi5. Achieves 10+ Hz map updates with 360-degree LiDAR scans.
2. **Indoor Accuracy** – Sub-centimeter accuracy in structured indoor environments. Excellent performance with the RPLiDAR C1's 0.72-degree angular resolution.
3. **Native ROS 2 Support** – First-class ROS 2 Humble/Jazzy support. Developed specifically for Nav2 integration.
4. **Flexible Operation Modes:**
 - **mapping** – Create new maps (online SLAM)
 - **localization** – Localize in existing maps (AMCL-like)
 - **lifelong** – Continuous mapping with map updates
5. **Serialization** – Maps can be saved/loaded in multiple formats (PGM, YAML, binary).

5.3 SLAM Toolbox Configuration

```

1 slam_toolbox:
2   ros__parameters:
3     # Solver parameters
4     solver_plugin: solver_plugins::CeresSolver
5     ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
6     ceres_preconditioner: SCHUR_JACOBI

```

```

7   ceres_trust_strategy: LEVENBERG_MARQUARDT
8   ceres_dogleg_type: TRADITIONAL_DOGLEG
9   ceres_loss_function: None
10
11  # Frame IDs
12  odom_frame: odom
13  map_frame: map
14  base_frame: base_link
15  scan_topic: /scan
16  use_scan_matching: true
17  use_scan_barycenter: true
18
19  # Resolution and range
20  resolution: 0.05          # 5cm per cell (matches Nav2 costmap)
21  max_laser_range: 12.0     # RPLiDAR C1 max range
22  minimum_travel_distance: 0.3
23  minimum_travel_heading: 0.3
24
25  # Loop closure
26  do_loop_closing: true
27  loop_search_maximum_distance: 3.0
28  loop_match_minimum_chain_size: 3
29  loop_match_maximum_variance_coarse: 3.0
30  loop_match_minimum_response_coarse: 0.35
31  loop_match_minimum_response_fine: 0.45
32
33  # Performance tuning for RPi5
34  transform_publish_period: 0.05 # 20 Hz TF publish rate
35  map_update_interval: 0.5       # 2 Hz map updates (reduce CPU)
36  throttle_scans: 1              # Process every scan

```

Listing 3: slam_toolbox_params.yaml

5.4 Why Not Other Algorithms?

Cartographer Excellent loop closure but computationally expensive. Requires Lua configuration which adds complexity. Better suited for high-end compute platforms.

Hector SLAM Fast and lightweight but lacks loop closure entirely. Drift accumulates over time in longer sessions. Best for short-duration mapping only.

GMapping Particle filter approach with good accuracy but higher memory usage due to particle storage. ROS 2 port is community-maintained, not official.

5.5 Nav2 DWA Controller Parameters

The Dynamic Window Approach (DWA) local planner requires careful tuning for the STAR robot's physical characteristics:

Table 5: DWA Controller Parameters

Parameter	Value	Description
Velocity Limits		
max_vel_x	0.5 m/s	Maximum forward velocity
min_vel_x	-0.3 m/s	Maximum reverse velocity
max_vel_theta	1.0 rad/s	Maximum angular velocity
min_speed_xy	0.0 m/s	Minimum translational speed
Acceleration Limits		
acc_lim_x	1.0 m/s ²	Linear acceleration limit
acc_lim_theta	2.0 rad/s ²	Angular acceleration limit
decel_lim_x	-1.5 m/s ²	Linear deceleration limit
decel_lim_theta	-2.5 rad/s ²	Angular deceleration limit
Trajectory Scoring		
path_distance_bias	32.0	Weight for staying on global path
goal_distance_bias	24.0	Weight for approaching goal
occdist_scale	0.01	Weight for avoiding obstacles
Costmap		
inflation_radius	0.3 m	Obstacle inflation radius
cost_scaling_factor	3.0	Exponential decay rate

```

1 controller_server:
2   ros_parameters:
3     controller_frequency: 20.0   # Hz
4     FollowPath:
5       plugin: "dwb_core::DWBLocalPlanner"
6       # Velocity limits
7       max_vel_x: 0.5
8       min_vel_x: -0.3
9       max_vel_y: 0.0           # Differential drive - no lateral motion
10      min_vel_y: 0.0
11      max_vel_theta: 1.0
12      min_speed_xy: 0.0
13      max_speed_xy: 0.5
14      min_speed_theta: 0.0
15      # Acceleration limits
16      acc_lim_x: 1.0
17      acc_lim_y: 0.0
18      acc_lim_theta: 2.0
19      decel_lim_x: -1.5
20      decel_lim_y: 0.0
21      decel_lim_theta: -2.5
22      # Trajectory generation
23      vx_samples: 20
24      vy_samples: 1             # Differential drive
25      vtheta_samples: 40
26      sim_time: 1.5             # Trajectory simulation time
27      # Critics (trajectory scoring)
28      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
29               "GoalAlign", "PathAlign", "PathDist", "GoalDist"]

```

```
30   PathAlign.scale: 32.0
31   GoalAlign.scale: 24.0
32   PathDist.scale: 32.0
33   GoalDist.scale: 24.0
34   BaseObstacle.scale: 0.01
```

Listing 4: nav2_params.yaml (DWA section)

6 ML Inference Pipeline

The STAR robot uses machine learning for person detection as part of its safety system. This section documents the model selection and inference pipeline.

6.1 Model Selection

Table 6: Person Detection Model Comparison

Model	Size (MB)	mAP@0.5	RPi5 FPS	Quantization
YOLOv8n INT8	3.2	0.37	15-18	INT8
YOLOv5n FP16	7.5	0.35	8-10	FP16
MobileNet-SSD	4.8	0.31	12-15	INT8
EfficientDet-Lite0	4.4	0.33	10-12	INT8

YOLOv8n INT8 was selected for its optimal balance of accuracy and performance on the Raspberry Pi 5.

6.2 Model Architecture

- **Base Model:** YOLOv8n (nano variant)
- **Input Resolution:** 320x320 (reduced from 640x640 for speed)
- **Quantization:** INT8 post-training quantization via TensorFlow Lite
- **Classes:** Single class (person) for focused detection
- **Inference Backend:** TensorFlow Lite with XNNPACK delegate

6.3 Inference Pipeline

```

1 import cv2
2 import numpy as np
3 from tflite_runtime.interpreter import Interpreter
4
5 class PersonDetector:
6     def __init__(self, model_path: str = "yolov8n_int8.tflite"):
7         # Load TFLite model with XNNPACK delegate for ARM optimization
8         self.interpreter = Interpreter(
9             model_path=model_path,
10            num_threads=4 # Use all 4 Cortex-A76 cores
11        )
12        self.interpreter.allocate_tensors()
13
14        self.input_details = self.interpreter.get_input_details()
15        self.output_details = self.interpreter.get_output_details()
16        self.input_shape = self.input_details[0]['shape'][1:3] # (320,
17                                                                320)
18        # Detection thresholds

```

```

19     self.confidence_threshold = 0.5
20     self.nms_threshold = 0.4
21
22     def preprocess(self, frame: np.ndarray) -> np.ndarray:
23         """Resize and normalize input frame."""
24         resized = cv2.resize(frame, tuple(self.input_shape))
25         # INT8 quantization: scale to 0-255 range
26         normalized = resized.astype(np.uint8)
27         return np.expand_dims(normalized, axis=0)
28
29     def detect(self, frame: np.ndarray) -> list:
30         """Run inference and return person detections."""
31         input_data = self.preprocess(frame)
32         self.interpreter.set_tensor(
33             self.input_details[0]['index'],
34             input_data
35         )
36         self.interpreter.invoke()
37
38         # Parse YOLO output format
39         output = self.interpreter.get_tensor(
40             self.output_details[0]['index']
41         )
42         detections = self._parse_output(output, frame.shape)
43         return detections
44
45     def _parse_output(self, output: np.ndarray,
46                       original_shape: tuple) -> list:
47         """Convert YOLO output to bounding boxes."""
48         detections = []
49         h, w = original_shape[:2]
50         scale_x, scale_y = w / self.input_shape[1], h /
51             self.input_shape[0]
52
53         for detection in output[0]:
54             confidence = detection[4]
55             if confidence < self.confidence_threshold:
56                 continue
57
58             # Extract bounding box (center format to corner format)
59             cx, cy, bw, bh = detection[:4]
60             x1 = int((cx - bw / 2) * scale_x)
61             y1 = int((cy - bh / 2) * scale_y)
62             x2 = int((cx + bw / 2) * scale_x)
63             y2 = int((cy + bh / 2) * scale_y)
64
65             detections.append({
66                 'bbox': (x1, y1, x2, y2),
67                 'confidence': float(confidence),
68                 'class': 'person'
69             })
70
71         # Apply non-maximum suppression
72         return self._nms(detections)

```

```

72
73     def _nms(self, detections: list) -> list:
74         """Non-maximum suppression to remove overlapping boxes."""
75         if not detections:
76             return []
77
78         boxes = np.array([d['bbox'] for d in detections])
79         scores = np.array([d['confidence'] for d in detections])
80
81         indices = cv2.dnn.NMSBoxes(
82             boxes.tolist(),
83             scores.tolist(),
84             self.confidence_threshold,
85             self.nms_threshold
86         )
87
88         return [detections[i] for i in indices.flatten()]

```

Listing 5: person_detector.py

6.4 Performance Optimization

1. **XNNPACK Delegate** – ARM-optimized SIMD operations for 2-3x speedup over default TFLite.
2. **Multi-threading** – Utilize all 4 Cortex-A76 cores via `num_threads=4`.
3. **Reduced Resolution** – 320x320 input (vs 640x640) reduces computation by 4x with minimal accuracy loss for person detection.
4. **Frame Skipping** – Process every 2nd frame (15 FPS effective) to maintain system responsiveness.
5. **ROI Processing** – In localization mode, only process regions where persons were previously detected.

6.5 Integration with Safety System

The person detector feeds into the CV Safety System (Section 7):

```

1  # In main control loop
2  detector = PersonDetector()
3  camera = cv2.VideoCapture(0)
4
5  while True:
6      ret, frame = camera.read()
7      if not ret:
8          continue
9
10     detections = detector.detect(frame)
11
12     for det in detections:
13         # Calculate distance from bounding box size
14         # (calibrated for known person height)

```

```
15     bbox_height = det['bbox'][3] - det['bbox'][1]
16     estimated_distance = estimate_distance(bbox_height)
17
18     # Trigger safety response based on distance
19     if estimated_distance < EMERGENCY_STOP_DISTANCE:
20         robot.emergency_stop()
21     elif estimated_distance < SLOW_ZONE_DISTANCE:
22         robot.reduce_speed(0.5) # 50% max speed
```

Listing 6: Integration with safety pipeline

7 CV Safety System

The computer vision safety system provides an additional layer of protection beyond LiDAR-based obstacle detection. It uses the USB camera and YOLOv8n person detector to identify humans and adjust robot behavior accordingly.

7.1 Safety Zones

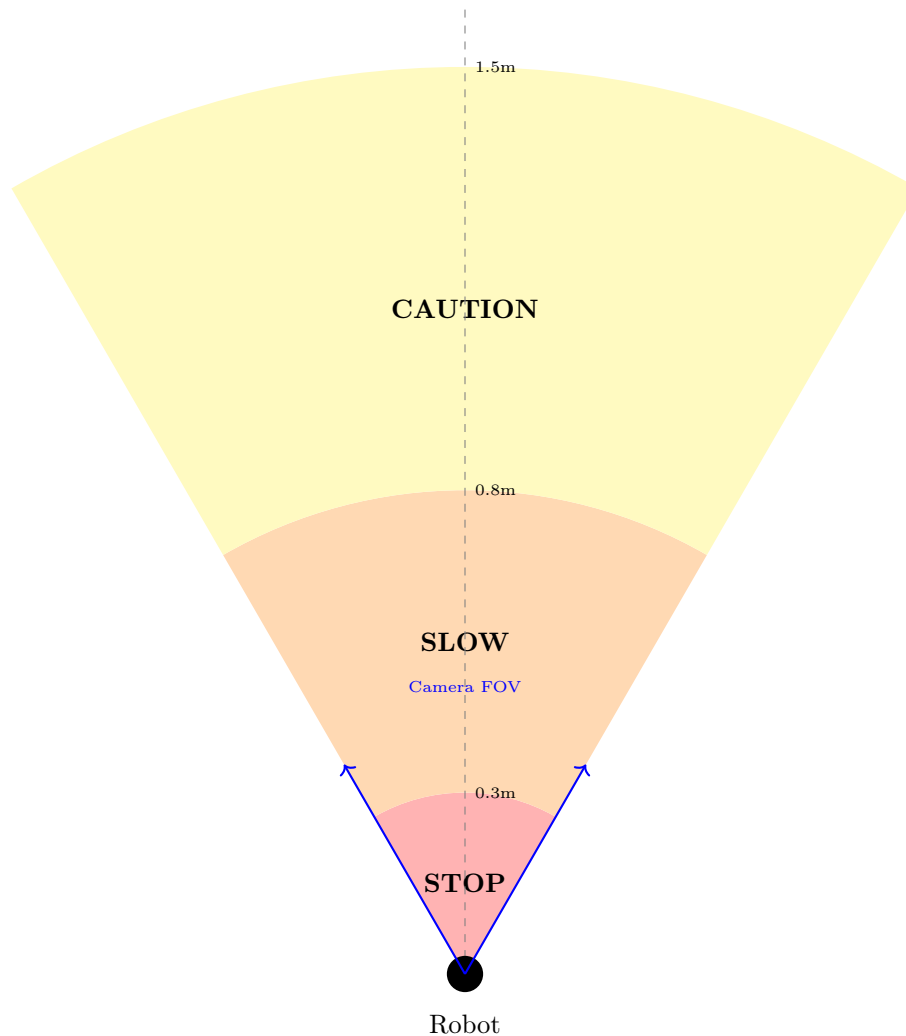


Figure 4: Safety Zone Distances (Top-Down View)

Table 7: Safety Zone Thresholds and Responses

Zone	Distance	Robot Response
Emergency Stop (Red)	$< 0.3 \text{ m}$	Immediate stop, hold position
Slow Zone (Orange)	$0.3 - 0.8 \text{ m}$	Reduce speed to 50% of max
Caution Zone (Yellow)	$0.8 - 1.5 \text{ m}$	Continue at current speed, log warning
Clear (Green)	$> 1.5 \text{ m}$	Normal operation

7.2 Distance Estimation

Since the USB camera provides 2D images without depth information, person distance is estimated using the detected bounding box height and a calibrated person height model:

```

1  # Calibration constants (adjust based on camera focal length and mounting)
2  FOCAL_LENGTH_PIXELS = 600 # Camera focal length in pixels
3  AVERAGE_PERSON_HEIGHT_M = 1.7 # Average adult height in meters
4  CAMERA_HEIGHT_M = 0.3 # Robot camera mounting height
5
6  def estimate_distance(bbox_height_pixels: int,
7                        frame_height: int) -> float:
8      """
9      Estimate distance to person using pinhole camera model.
10
11      distance = (real_height * focal_length) / image_height
12
13      Args:
14          bbox_height_pixels: Height of person bounding box in pixels
15          frame_height: Total frame height in pixels
16
17      Returns:
18          Estimated distance in meters
19      """
20      if bbox_height_pixels <= 0:
21          return float('inf')
22
23      # Pinhole camera model
24      distance = (AVERAGE_PERSON_HEIGHT_M * FOCAL_LENGTH_PIXELS) /
25                  bbox_height_pixels
26
27      # Apply height correction (camera is not at ground level)
28      # This is a simplification; more accurate models use full pose
29      estimation
30      distance = max(0.1, distance)
31
32      return distance
33
34  # Calibration procedure:
35  # 1. Place person at known distance (e.g., 1.0m, 2.0m, 3.0m)
36  # 2. Record bounding box heights
37  # 3. Calculate FOCAL_LENGTH_PIXELS = (bbox_height * distance) /
38  person_height
39  # 4. Average across multiple measurements

```

Listing 7: Distance estimation from bounding box

7.3 Safety State Machine

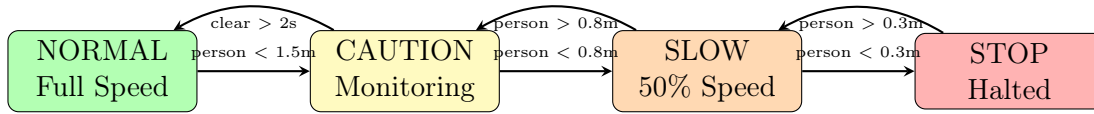


Figure 5: CV Safety State Machine

7.4 Integration with Navigation

The CV safety system overrides navigation commands when necessary:

```

1 from enum import Enum
2 from dataclasses import dataclass
3 from typing import Optional
4 import time
5
6 class SafetyState(Enum):
7     NORMAL = "normal"
8     CAUTION = "caution"
9     SLOW = "slow"
10    STOP = "stop"
11
12 @dataclass
13 class SafetyZones:
14     EMERGENCY_STOP: float = 0.3    # meters
15     SLOW_ZONE: float = 0.8         # meters
16     CAUTION_ZONE: float = 1.5     # meters
17     CLEAR_TIMEOUT: float = 2.0    # seconds before returning to normal
18
19 class SafetyController:
20     def __init__(self):
21         self.state = SafetyState.NORMAL
22         self.last_detection_time: Optional[float] = None
23         self.speed_multiplier = 1.0
24
25     def update(self, person_distance: Optional[float]) -> SafetyState:
26         """Update safety state based on nearest person distance."""
27         current_time = time.time()
28
29         if person_distance is None:
30             # No person detected
31             if self.last_detection_time is not None:
32                 elapsed = current_time - self.last_detection_time
33                 if elapsed > SafetyZones.CLEAR_TIMEOUT:
34                     self._transition_to(SafetyState.NORMAL)
35             return self.state
36
37         self.last_detection_time = current_time
38
39         # Determine appropriate state based on distance
40         if person_distance < SafetyZones.EMERGENCY_STOP:
41             self._transition_to(SafetyState.STOP)

```

```
42     elif person_distance < SafetyZones.SLOW_ZONE:
43         self._transition_to(SafetyState.SLOW)
44     elif person_distance < SafetyZones.CAUTION_ZONE:
45         self._transition_to(SafetyState.CAUTION)
46
47     return self.state
48
49     def _transition_to(self, new_state: SafetyState):
50         if new_state != self.state:
51             print(f"Safety state: {self.state.value} ->
52                   {new_state.value}")
53             self.state = new_state
54             self._update_speed_multiplier()
55
56     def _update_speed_multiplier(self):
57         multipliers = {
58             SafetyState.NORMAL: 1.0,
59             SafetyState.CAUTION: 1.0,
60             SafetyState.SLOW: 0.5,
61             SafetyState.STOP: 0.0,
62         }
63         self.speed_multiplier = multipliers[self.state]
64
65     def apply_to_velocity(self, cmd_vel: tuple) -> tuple:
66         """Apply safety multiplier to velocity command."""
67         linear, angular = cmd_vel
68         return (linear * self.speed_multiplier,
69               angular * self.speed_multiplier)
```

Listing 8: SafetyController class

8 Battery Management

The STAR robot implements a multi-threshold battery management system to ensure safe operation and prevent damage from deep discharge.

8.1 Battery State Thresholds

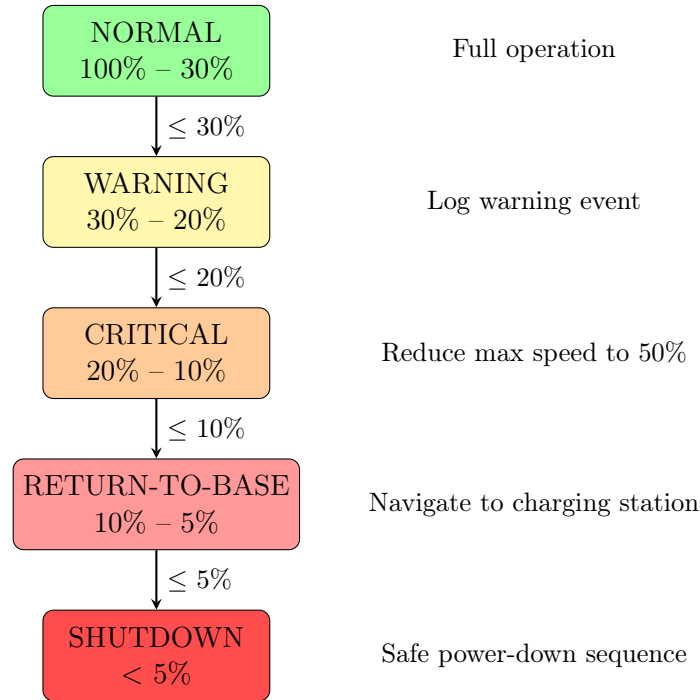


Figure 6: Battery State Machine

Table 8: Battery Management Thresholds

Level	State	Action
100% – 30%	Normal	Full operation, no restrictions
30%	Warning	Log battery warning event to telemetry
20%	Critical	Reduce maximum speed to 50%
10%	Return-to-Base	Abort current task, navigate to charging station
5%	Emergency	Initiate safe shutdown sequence

8.2 Battery Monitoring

Battery voltage and state-of-charge are monitored via the ESP32's ADC with filtering:

```

1 #include <esp_adc_cal.h>
2
3 #define BATTERY_ADC_CHANNEL ADC1_CHANNEL_6 // GPIO34
4 #define BATTERY_SAMPLES 16 // Averaging samples
5 #define VOLTAGE_DIVIDER_RATIO 4.0 // R1=30k, R2=10k
6 #define BATTERY_MIN_V 10.0 // 3S LiPo empty

```

```

7  #define BATTERY_MAX_V          12.6          // 3S LiPo full
8
9  typedef enum {
10     BATTERY_NORMAL,
11     BATTERY_WARNING,
12     BATTERY_CRITICAL,
13     BATTERY_RTb,
14     BATTERY_SHUTDOWN
15 } battery_state_t;
16
17 static esp_adc_cal_characteristics_t adc_chars;
18 static battery_state_t current_state = BATTERY_NORMAL;
19
20 void battery_init(void) {
21     adc1_config_width(ADC_WIDTH_BIT_12);
22     adc1_config_channel_atten(BATTERY_ADC_CHANNEL, ADC_ATTEN_DB_11);
23     esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11,
24                             ADC_WIDTH_BIT_12, 1100, &adc_chars);
25 }
26
27 float battery_read_voltage(void) {
28     uint32_t adc_sum = 0;
29
30     // Multi-sample averaging for noise reduction
31     for (int i = 0; i < BATTERY_SAMPLES; i++) {
32         adc_sum += adc1_get_raw(BATTERY_ADC_CHANNEL);
33     }
34     uint32_t adc_avg = adc_sum / BATTERY_SAMPLES;
35
36     // Convert to voltage
37     uint32_t voltage_mv = esp_adc_cal_raw_to_voltage(adc_avg, &adc_chars);
38
39     return (voltage_mv / 1000.0f) * VOLTAGE_DIVIDER_RATIO;
40 }
41
42 uint8_t battery_get_percentage(float voltage) {
43     // Linear approximation (for LiPo, use lookup table for accuracy)
44     float pct = (voltage - BATTERY_MIN_V) / (BATTERY_MAX_V -
45         BATTERY_MIN_V);
46     pct = fmaxf(0.0f, fminf(1.0f, pct));
47     return (uint8_t)(pct * 100.0f);
48 }
49
50 battery_state_t battery_update_state(uint8_t percentage) {
51     if (percentage <= 5) {
52         current_state = BATTERY_SHUTDOWN;
53     } else if (percentage <= 10) {
54         current_state = BATTERY_RTb;
55     } else if (percentage <= 20) {
56         current_state = BATTERY_CRITICAL;
57     } else if (percentage <= 30) {
58         current_state = BATTERY_WARNING;
59     } else {
60         current_state = BATTERY_NORMAL;
61     }
62 }

```

```
60     }  
61     return current_state;  
62 }
```

Listing 9: Battery monitoring (ESP32)

8.3 Safe Shutdown Sequence

When battery reaches critical level (5%), the system initiates a controlled shutdown:

1. **Stop Motors** – Immediate motor disable, engage brakes if available
2. **Log Position** – Save current map position to persistent storage
3. **Notify User** – Send critical battery notification via GraphQL subscription
4. **Save State** – Persist navigation state, waypoints, and settings to flash
5. **Shutdown Sequence** – Graceful ROS 2 node shutdown, then Linux poweroff

9 Repository Structure

9.1 Directory Layout

```

1 /STAR/
2 |-- ESP32-Firmware/           # Existing - keep as-is
3 |-- star-pi5-os/             # Existing - add gateway to Buildroot
4 |-- Schematic/               # Existing - hardware designs
5 |-- Locked_Inc_FTP/          # Existing - documentation
6 |-- docs/                   # Existing - documentation
7 |
8 |-- star-gateway/            # NEW: Spring Boot Gateway
9 |   |-- build.gradle.kts
10 |   |-- settings.gradle.kts
11 |   |-- src/
12 |   |   +-- main/
13 |   |       |-- kotlin/com/star/gateway/
14 |   |           |-- StarGatewayApplication.kt
15 |   |           |-- config/
16 |   |               |-- GraphQLConfig.kt
17 |   |               |-- GrpcConfig.kt
18 |   |               +-- CorsConfig.kt
19 |   |               |-- graphql/
20 |   |                   |-- TelemetryController.kt
21 |   |                   |-- SettingsController.kt
22 |   |                   +-- NavigationController.kt
23 |   |               |-- grpc/
24 |   |                   +-- MotorControlGrpcService.kt
25 |   |               |-- service/
26 |   |                   |-- MotorControlService.kt
27 |   |                   |-- TelemetryService.kt
28 |   |                   |-- NavigationService.kt
29 |   |                   +-- SensorFusionService.kt
30 |   |               |-- hardware/
31 |   |                   |-- Esp32SpiAdapter.kt
32 |   |                   |-- Esp32TcpAdapter.kt
33 |   |                   |-- RplidarAdapter.kt
34 |   |                   +-- Ros2Bridge.kt
35 |   |               +-- model/
36 |   |                   |-- TelemetryData.kt
37 |   |                   |-- MotorCommand.kt
38 |   |                   +-- NavigationState.kt
39 |   |               |-- proto/
40 |   |                   +-- motor_control.proto
41 |   |               +-- resources/
42 |   |                   |-- application.yml
43 |   |                   +-- graphql/
44 |   |                       +-- schema.graphqls
45 |   +-- deployment/
46 |       |-- star-gateway.service    # Systemd unit
47 |       +-- avahi/
48 |           +-- star-robot.service  # mDNS config
49 |

```

```

50 |-- star-ui/                                # NEW: PWA (moved from UI-Demo)
51 |   |-- package.json
52 |   |-- vite.config.js
53 |   |-- public/
54 |   |   |-- manifest.json
55 |   |   +-- sw.js
56 |   |-- src/
57 |   |   |-- main.jsx
58 |   |   |-- App.jsx
59 |   |   |-- components/                    # From UI-Demo
60 |   |   |-- pages/                         # From UI-Demo
61 |   |   |-- hooks/
62 |   |   |   |-- useRobotConnection.js
63 |   |   |   |-- useTelemetry.js
64 |   |   |   +-- useMotorControl.js
65 |   |   |-- graphql/
66 |   |   |   |-- client.js
67 |   |   |   |-- queries.js
68 |   |   |   |-- mutations.js
69 |   |   |   +-- subscriptions.js
70 |   |   |-- grpc/
71 |   |   |   |-- client.js
72 |   |   |   +-- motor_control_pb.js
73 |   |   |-- service-worker/
74 |   |   |   |-- sw.js
75 |   |   |   +-- offline-cache.js
76 |   |   +-- storage/
77 |   |       +-- indexedDB.js
78 |   +-- workbox-config.js
79 |
80 +-- proto/                                # SHARED: Protobuf definitions
81 |   +-- star/
82 |       |-- motor_control.proto
83 |       |-- telemetry.proto
84 |       +-- navigation.proto

```

Listing 10: Complete Repository Structure

10 Spring Boot Gateway Implementation

10.1 Build Configuration

```
1 plugins {
2     kotlin("jvm") version "1.9.22"
3     kotlin("plugin.spring") version "1.9.22"
4     id("org.springframework.boot") version "3.2.4"
5     id("io.spring.dependency-management") version "1.1.4"
6     id("com.google.protobuf") version "0.9.4"
7 }
8
9 java {
10     sourceCompatibility = JavaVersion.VERSION_17
11 }
12
13 repositories {
14     mavenCentral()
15 }
16
17 dependencies {
18     // Spring Boot Core
19     implementation("org.springframework.boot:spring-boot-starter-webflux")
20     implementation("org.springframework.boot:spring-boot-starter-actuator")
21
22     // GraphQL
23     implementation("org.springframework.boot:spring-boot-starter-graphql")
24
25     // gRPC
26     implementation("net.devh:grpc-spring-boot-starter:3.1.0.RELEASE")
27     implementation("io.grpc:grpc-kotlin-stub:1.4.1")
28     implementation("io.grpc:grpc-protobuf:1.60.1")
29     implementation("com.google.protobuf:protobuf-kotlin:3.25.2")
30
31     // Kotlin Coroutines
32     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
33     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.8.0")
34     implementation("io.projectreactor.kotlin:reactor-kotlin-extensions")
35
36     // Hardware Access (Pi4J for SPI/GPIO)
37     // Pi4J 2.6+ required for Raspberry Pi 5 (RP1 chip support via GpioD)
38     implementation("com.pi4j:pi4j-core:2.6.1")
39     implementation("com.pi4j:pi4j-plugin-raspberrypi:2.6.1")
40     implementation("com.pi4j:pi4j-plugin-gpiod:2.6.1") // Required for
41     // RPi5
42     implementation("com.pi4j:pi4j-plugin-linuxfs:2.6.1")
43
44     // Metrics
45     implementation("io.micrometer:micrometer-registry-prometheus")
46
47     // JSON Processing
48     implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
```

```

49  // Kotlin Reflection
50  implementation("org.jetbrains.kotlin:kotlin-reflect")
51
52  // Testing
53  testImplementation("org.springframework.boot:spring-boot-starter-test")
54  testImplementation("io.projectreactor:reactor-test")
55 }
56
57 protobuf {
58     protoc {
59         artifact = "com.google.protobuf:protoc:3.25.2"
60     }
61     plugins {
62         id("grpc") {
63             artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
64         }
65         id("grpc_kotlin") {
66             artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
67         }
68     }
69     generateProtoTasks {
70         all().forEach {
71             it.plugins {
72                 id("grpc")
73                 id("grpc_kotlin")
74             }
75             it.builtins {
76                 id("kotlin")
77             }
78         }
79     }
80 }
81
82 tasks.withType<org.jetbrains.kotlin.gradle.tasks.KotlinCompile> {
83     kotlinOptions {
84         freeCompilerArgs += "-Xjsr305=strict"
85         jvmTarget = "17"
86     }
87 }

```

Listing 11: build.gradle.kts

10.2 Application Configuration

```

1  spring:
2    application:
3      name: star-gateway
4    graphql:
5      graphiql:
6        enabled: true
7        path: /graphiql
8      websocket:
9        path: /graphql-ws

```

```
10     schema:
11       locations: classpath:graphql/
12     main:
13       banner-mode: off
14
15     server:
16       port: 8080
17
18     grpc:
19       server:
20         port: 9090
21
22     # Hardware configuration
23     star:
24       esp32:
25         spi:
26           enabled: true
27           bus: 0
28           chip-select: 0
29           speed-hz: 10000000 # 10 MHz for motor control
30           mode: 0
31         tcp:
32           enabled: true
33           host: 192.168.4.2 # ESP32 IP on AP network
34           port: 5000
35           timeout-ms: 100
36           retry-attempts: 3
37       lidar:
38         serial-port: /dev/ttyAMA0
39         baud-rate: 460800
40         protocol: SLAMTEC
41         scan-rate-hz: 10
42
43     # Actuator endpoints for health checks
44     management:
45       endpoints:
46         web:
47           exposure:
48             include: health,prometheus,info,metrics
49       endpoint:
50         health:
51           show-details: always
52       metrics:
53         tags:
54           application: star-gateway
55
56     logging:
57       level:
58         com.star.gateway: DEBUG
59         org.springframework.graphql: INFO
```

Listing 12: application.yml

10.3 Six-Layer Software Architecture

The STAR system follows a strict six-layer architecture spanning from the remote UI to embedded firmware:

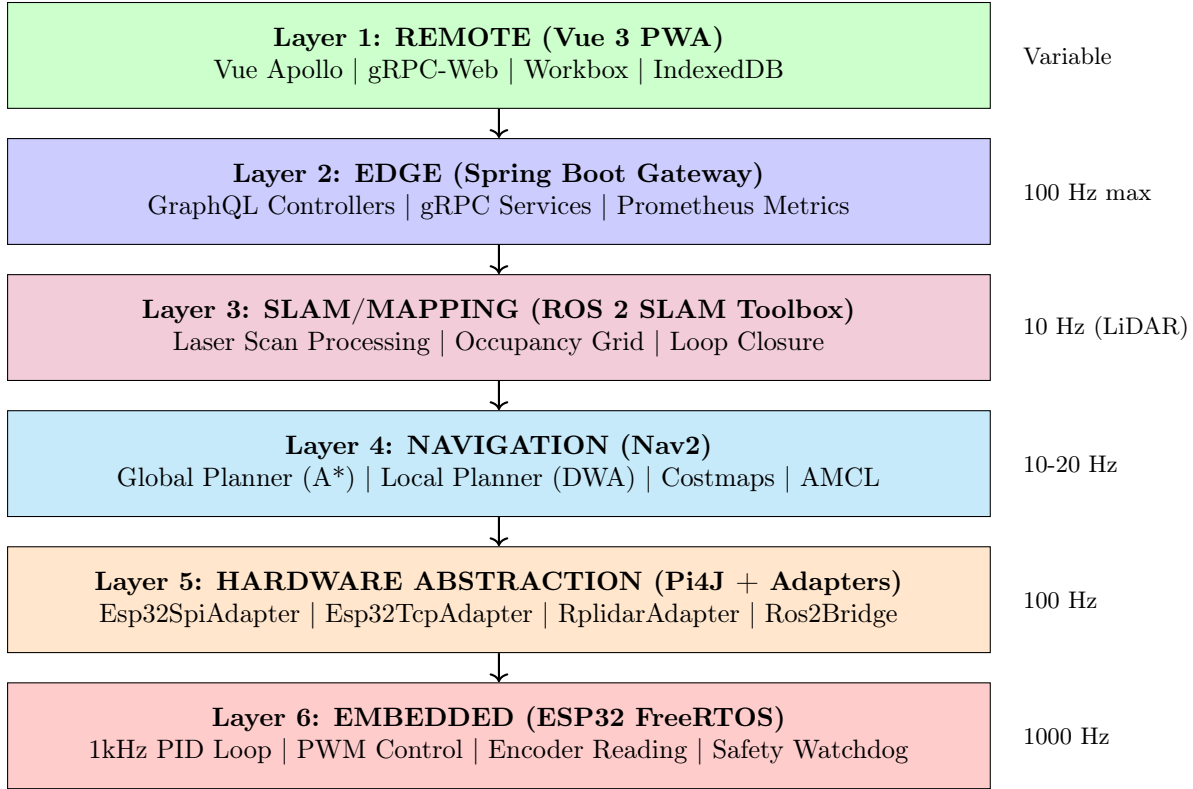


Figure 7: Six-Layer Software Architecture with Update Frequencies

10.3.1 Layer Responsibilities

Layer 1 - Remote (Vue 3 PWA)

User interface running in browser or as installed PWA. Handles user input, displays telemetry, provides offline capability via service workers and IndexedDB.

Layer 2 - Edge (Spring Boot Gateway)

API gateway running on RPi5. Translates between web protocols (GraphQL/gRPC) and robot internals. Manages authentication, rate limiting, and metrics.

Layer 3 - SLAM/Mapping (ROS 2 SLAM Toolbox)

Processes LiDAR scans to build occupancy grid maps. Handles loop closure detection and map optimization. Updates at LiDAR scan rate (10 Hz).

Layer 4 - Navigation (Nav2)

Path planning and obstacle avoidance. Global planner (A*) runs at 10 Hz, local planner (DWA) runs at 20 Hz. Publishes velocity commands to Layer 5.

Layer 5 - Hardware Abstraction (Pi4J + Adapters)

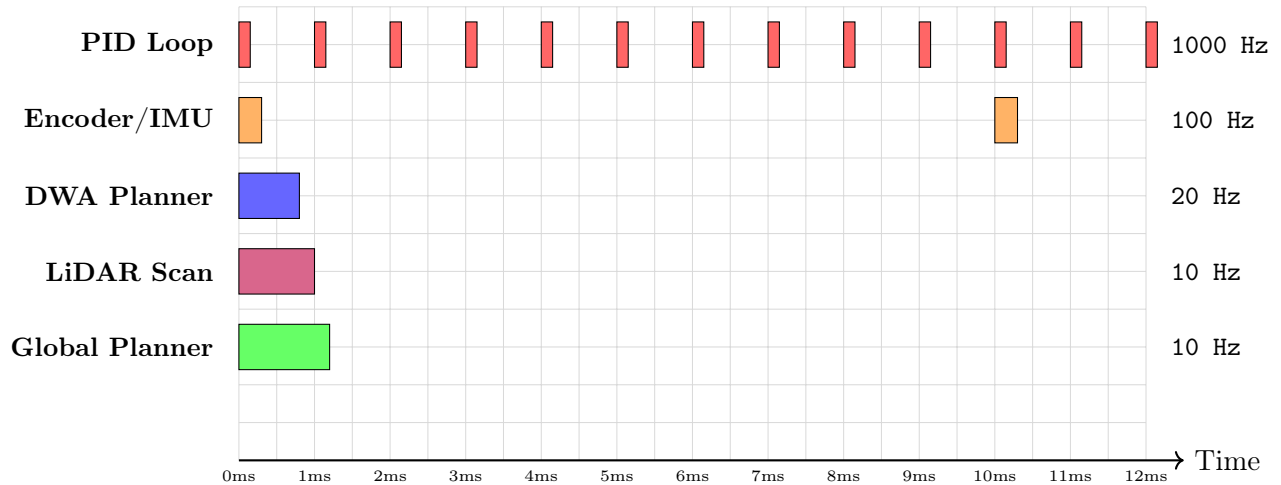
Interfaces between ROS 2 and physical hardware. Communicates with ESP32 via SPI (primary) and TCP (fallback). Reads LiDAR via UART (rplidar_ros2 driver).

Layer 6 - Embedded (ESP32 FreeRTOS)

Real-time motor control at 1 kHz. Runs PID loops, reads encoders, generates PWM signals. Maintains safe operation even if higher layers fail.

10.3.2 Control Timing Hierarchy

The STAR robot operates with a multi-rate control hierarchy, where each layer runs at a frequency appropriate to its function:



Note: Diagram shows 12ms window. Lower frequency tasks extend beyond visible range.

Figure 8: Control Timing Hierarchy (12ms Window)

Table 9: Control Loop Frequencies and Responsibilities

Control Loop	Frequency	Period	Processor
PID Motor Control	1000 Hz	1 ms	ESP32 (FreeRTOS)
Encoder/IMU Sampling	100 Hz	10 ms	ESP32 (FreeRTOS)
DWA Local Planner	20 Hz	50 ms	RPi5 (Nav2)
LiDAR Scan Processing	10 Hz	100 ms	RPi5 (ROS 2)
Global Path Planner	10 Hz	100 ms	RPi5 (Nav2)
Telemetry Publishing	10 Hz	100 ms	RPi5 (Spring Boot)
Map Updates	2 Hz	500 ms	RPi5 (SLAM Toolbox)

Timing Design Rationale:

- **1000 Hz PID** – Required for smooth motor control and quick response to disturbances. FreeRTOS ensures deterministic timing.
- **100 Hz Sensor Sampling** – Captures high-frequency vibrations and provides accurate velocity estimates via differentiation.
- **20 Hz DWA** – Fast enough for obstacle avoidance at max velocity ($0.5 \text{ m/s} = 2.5\text{cm}$ per cycle), slow enough to not overload CPU.

- **10 Hz LiDAR** – Fixed by RPLiDAR C1 hardware. Sufficient for indoor navigation at walking speeds.
- **10 Hz Global Planner** – Path replanning happens infrequently; 10 Hz allows quick re-routing while minimizing CPU load.

10.4 GraphQL Schema

```
1  # Query operations - read-only data fetching
2  type Query {
3      telemetry: Telemetry!
4      systemStatus: SystemStatus!
5      settings: Settings!
6      navigationState: NavigationState!
7      telemetryHistory(limit: Int = 100): [TelemetrySnapshot!]!
8  }
9
10 # Mutation operations - state changes
11 type Mutation {
12     setMode(mode: RobotMode!): Boolean!
13     emergencyStop: Boolean!
14     setSettings(input: SettingsInput!): Settings!
15     addWaypoint(x: Float!, y: Float!): Waypoint!
16     clearWaypoints: Boolean!
17     startNavigation: Boolean!
18     stopNavigation: Boolean!
19 }
20
21 # Subscription operations - real-time streams
22 type Subscription {
23     telemetryStream: Telemetry!
24     lidarScan: LidarScan!
25     eventLog: Event!
26 }
27
28 # Core telemetry data structure
29 type Telemetry {
30     imu: ImuData!
31     gps: GpsData!
32     battery: Float!
33     wifiSignal: Int!
34     cpuUsage: Float!
35     temperature: Float!
36     motorLoad: Float!
37     timestamp: String!
38 }
39
40 type ImuData {
41     pitch: Float!
42     roll: Float!
43     yaw: Float!
44     velocity: Float!
45     accelerationX: Float!
```

```
46     accelerationY: Float!
47     accelerationZ: Float!
48 }
49
50 type GpsData {
51     latitude: Float!
52     longitude: Float!
53     altitude: Float!
54 }
55
56 type LidarScan {
57     points: [LidarPoint!]!
58     timestamp: String!
59 }
60
61 type LidarPoint {
62     angle: Float!
63     distance: Float!
64     intensity: Float!
65 }
66
67 type SystemStatus {
68     connectionStatus: ConnectionStatus!
69     mode: RobotMode!
70     esp32Connected: Boolean!
71     lidarConnected: Boolean!
72     rosConnected: Boolean!
73 }
74
75 type NavigationState {
76     currentPosition: Position!
77     path: [Position!]!
78     waypoints: [Waypoint!]!
79     isNavigating: Boolean!
80 }
81
82 type Position {
83     x: Float!
84     y: Float!
85     heading: Float!
86 }
87
88 type Waypoint {
89     id: ID!
90     x: Float!
91     y: Float!
92     label: String
93 }
94
95 type Event {
96     timestamp: String!
97     message: String!
98     type: EventType!
99 }
```

```

100
101 type Settings {
102     maxSpeed: Float!
103     safetyLockEnabled: Boolean!
104     autonomousEnabled: Boolean!
105     lidarEnabled: Boolean!
106 }
107
108 input SettingsInput {
109     maxSpeed: Float
110     safetyLockEnabled: Boolean
111     autonomousEnabled: Boolean
112     lidarEnabled: Boolean
113 }
114
115 enum RobotMode {
116     IDLE
117     MANUAL
118     AUTONOMOUS
119     MAPPING
120     EMERGENCY_STOP
121 }
122
123 enum ConnectionStatus {
124     CONNECTED
125     DISCONNECTED
126     CONNECTING
127     ERROR
128 }
129
130 enum EventType {
131     INFO
132     SUCCESS
133     WARNING
134     ERROR
135 }

```

Listing 13: schema.graphqls

10.5 gRPC Protobuf Definition

```

1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_outer_classname = "MotorControlProto";
7 option java_multiple_files = true;
8
9 // Motor control service for low-latency commands
10 service MotorControl {
11     // Unary RPCs (single request-response)
12     rpc SetVelocity(VelocityCommand) returns (CommandResponse);

```

```

13  rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
14  rpc SetMotorPower(MotorPowerCommand) returns (CommandResponse);
15
16  // Server streaming (continuous encoder feedback)
17  rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
18 }
19
20 // Velocity command for differential drive
21 message VelocityCommand {
22     float left_velocity = 1;    // m/s, range: -2.0 to 2.0
23     float right_velocity = 2;   // m/s, range: -2.0 to 2.0
24     uint64 timestamp = 3;       // Unix timestamp in milliseconds
25 }
26
27 // Direct motor power control
28 message MotorPowerCommand {
29     int32 motor_id = 1;         // Motor index: 0-3
30     float power = 2;            // Power level: -1.0 to 1.0
31 }
32
33 // Standard command response
34 message CommandResponse {
35     bool success = 1;           // True if command executed
36     string message = 2;         // Human-readable status
37     uint64 latency_us = 3;      // Round-trip latency in microseconds
38 }
39
40 // Real-time encoder data
41 message EncoderData {
42     int32 motor_id = 1;         // Motor index: 0-3
43     int64 ticks = 2;            // Absolute encoder position
44     float velocity = 3;         // Current velocity in m/s
45     uint64 timestamp = 4;       // Unix timestamp in milliseconds
46 }
47
48 // Empty request placeholder
49 message EmptyRequest {}

```

Listing 14: motor_control.proto

10.6 Service Implementations

10.6.1 Motor Control Service

```

1  package com.star.gateway.service
2
3  import com.star.gateway.hardware.Esp32SpiAdapter
4  import com.star.gateway.hardware.Esp32TcpAdapter
5  import com.star.gateway.model.CommandResult
6  import io.micrometer.core.instrument.MeterRegistry
7  import io.micrometer.core.instrument.Timer
8  import kotlinx.coroutines.async
9  import kotlinx.coroutines.coroutineScope

```

```

10 import org.slf4j.LoggerFactory
11 import org.springframework.stereotype.Service
12 import java.util.concurrent.TimeUnit
13
14 @Service
15 class MotorControlService(
16     private val esp32SpiAdapter: Esp32SpiAdapter,
17     private val esp32TcpAdapter: Esp32TcpAdapter,
18     private val meterRegistry: MeterRegistry
19 ) {
20     private val logger = LoggerFactory.getLogger(javaClass)
21
22     // Metrics
23     private val commandCounter =
24         meterRegistry.counter("motor.commands.total")
25     private val spiErrorCounter =
26         meterRegistry.counter("motor.spi.errors")
27     private val tcpFallbackCounter =
28         meterRegistry.counter("motor.tcp.fallbacks")
29     private val latencyTimer: Timer =
30         Timer.builder("motor.command.latency")
31             .description("Motor command round-trip latency")
32             .register(meterRegistry)
33
34     /**
35      * Send velocity command to ESP32.
36      * Primary: SPI (lowest latency)
37      * Fallback: TCP (if SPI fails)
38      */
39     suspend fun setVelocity(left: Float, right: Float): CommandResult {
40         val startTime = System.nanoTime()
41
42         return try {
43             val result = esp32SpiAdapter.sendVelocityCommand(left, right)
44             commandCounter.increment()
45             latencyTimer.record(System.nanoTime() - startTime,
46                               TimeUnit.NANOSECONDS)
47             result
48         } catch (e: Exception) {
49             logger.warn("SPI failed, falling back to TCP: ${e.message}")
50             spiErrorCounter.increment()
51             tcpFallbackCounter.increment()
52
53             try {
54                 esp32TcpAdapter.sendVelocityCommand(left, right)
55             } catch (tcpException: Exception) {
56                 logger.error("TCP fallback also failed:
57                             ${tcpException.message}")
58                 CommandResult.failure("Both SPI and TCP communication
59                                     failed")
60             }
61         }
62     }
63 }

```

```

57  /**
58   * Emergency stop - send to BOTH channels simultaneously for
59   * redundancy.
60   * Returns success if either channel succeeds.
61   */
62  suspend fun emergencyStop(): CommandResult {
63      logger.warn("EMERGENCY STOP triggered")
64
65      return coroutineScope {
66          val spiDeferred = async {
67              runCatching { esp32SpiAdapter.emergencyStop() }
68          }
69          val tcpDeferred = async {
70              runCatching { esp32TcpAdapter.emergencyStop() }
71          }
72
73          val spiResult = spiDeferred.await()
74          val tcpResult = tcpDeferred.await()
75
76          when {
77              spiResult.isSuccess && tcpResult.isSuccess -> {
78                  logger.info("Emergency stop confirmed on both
79                      channels")
80                  CommandResult.success("Emergency stop executed on
81                      both channels")
82              }
83              spiResult.isSuccess || tcpResult.isSuccess -> {
84                  logger.warn("Emergency stop partially successful")
85                  CommandResult.success("Emergency stop executed
86                      (partial)")
87              }
88              else -> {
89                  logger.error("Emergency stop FAILED on both
90                      channels!")
91                  CommandResult.failure("Emergency stop failed on both
92                      channels")
93              }
94          }
95      }
96  }
97
98  /**
99   * Get current connection status for both channels.
100  */
101  fun getConnectionStatus(): Map<String, Boolean> {
102      return mapOf(
103          "spi" to esp32SpiAdapter.isConnected(),
104          "tcp" to esp32TcpAdapter.isConnected()
105      )
106  }
107
108  }

```

Listing 15: MotorControlService.kt

10.6.2 ESP32 SPI Adapter

```

1 package com.star.gateway.hardware
2
3 import com.pi4j.Pi4J
4 import com.pi4j.context.Context
5 import com.pi4j.io.spi.Spi
6 import com.pi4j.io.spi.SpiMode
7 import com.star.gateway.model.CommandResult
8 import org.slf4j.LoggerFactory
9 import org.springframework.beans.factory.annotation.Value
10 import org.springframework.stereotype.Component
11 import java.nio.ByteBuffer
12 import java.nio.ByteOrder
13 import javax.annotation.PostConstruct
14 import javax.annotation.PreDestroy
15
16 @Component
17 class Esp32SpiAdapter(
18     @Value("${star.esp32.spi.enabled:true}")
19     private val enabled: Boolean,
20
21     @Value("${star.esp32.spi.bus:0}")
22     private val spiBus: Int,
23
24     @Value("${star.esp32.spi.chip-select:0}")
25     private val chipSelect: Int,
26
27     @Value("${star.esp32.spi.speed-hz:10000000}") // 10 MHz default
28     private val speedHz: Int,
29
30     @Value("${star.esp32.spi.mode:0}")
31     private val spiModeValue: Int
32 ) {
33     private val logger = LoggerFactory.getLogger(javaClass)
34
35     private var pi4j: Context? = null
36     private var spi: Spi? = null
37     private var connected = false
38
39     // SPI Command bytes
40     companion object {
41         const val CMD_SET_VELOCITY: Byte = 0x01
42         const val CMD_EMERGENCY_STOP: Byte = 0x02
43         const val CMD_GET_ENCODERS: Byte = 0x03
44         const val CMD_SET_MOTOR: Byte = 0x04
45         const val CMD_PING: Byte = 0x05
46
47         const val RESPONSE_OK: Byte = 0x00
48         const val RESPONSE_ERROR: Byte = 0x01
49     }
50
51     @PostConstruct
52     fun initialize() {

```

```

53     if (!enabled) {
54         logger.info("SPI adapter disabled by configuration")
55         return
56     }
57
58     try {
59         pi4j = Pi4J.newAutoContext()
60
61         val spiConfig = Spi.newConfigBuilder(pi4j)
62             .id("esp32-spi")
63             .name("ESP32 Motor Controller")
64             .bus(spiBus)
65             .chipSelect(chipSelect)
66             .baud(speedHz)
67             .mode(SpiMode.entries[spiModeValue])
68             .build()
69
70         spi = pi4j?.create(spiConfig)
71         connected = testConnection()
72
73         logger.info("SPI adapter initialized: bus=$spiBus,
74             cs=$chipSelect, speed=$speedHz Hz")
75     } catch (e: Exception) {
76         logger.error("Failed to initialize SPI adapter: ${e.message}")
77         connected = false
78     }
79
80     @PreDestroy
81     fun shutdown() {
82         spi?.close()
83         pi4j?.shutdown()
84         logger.info("SPI adapter shutdown complete")
85     }
86
87     /**
88      * Send velocity command.
89      * Packet format: [CMD_SET_VELOCITY (1), left_velocity (4),
90      * right_velocity (4)]
91      * Response: [status (1)]
92      */
93     suspend fun sendVelocityCommand(left: Float, right: Float):
94         CommandResult {
95         CommandResult {
96             if (!enabled || spi == null) {
97                 throw IllegalStateException("SPI not initialized")
98             }
99
100             val buffer = ByteBuffer.allocate(9)
101                 .order(ByteOrder.LITTLE_ENDIAN)
102                 .put(CMD_SET_VELOCITY)
103                 .putFloat(left)
104                 .putFloat(right)
105
106             val txData = buffer.array()

```

```

104     val rxData = ByteArray(2)
105
106     synchronized(this) {
107         spi!!.transfer(txData, rxData)
108     }
109
110     return if (rxData[0] == RESPONSE_OK) {
111         CommandResult.success("Velocity set: left=$left,
112                                right=$right")
113     } else {
114         CommandResult.failure("ESP32 error code: ${rxData[0]}")
115     }
116
117     /**
118     * Send emergency stop command.
119     * Packet format: [CMD_EMERGENCY_STOP (1)]
120     * Response: [status (1)]
121     */
122     suspend fun emergencyStop(): CommandResult {
123         if (!enabled || spi == null) {
124             throw IllegalStateException("SPI not initialized")
125         }
126
127         val txData = byteArrayOf(CMD_EMERGENCY_STOP)
128         val rxData = ByteArray(2)
129
130         synchronized(this) {
131             spi!!.transfer(txData, rxData)
132         }
133
134         return if (rxData[0] == RESPONSE_OK) {
135             CommandResult.success("Emergency stop executed")
136         } else {
137             CommandResult.failure("Emergency stop failed: ${rxData[0]}")
138         }
139     }
140
141     /**
142     * Test SPI connection with ping command.
143     */
144     private fun testConnection(): Boolean {
145         return try {
146             val txData = byteArrayOf(CMD_PING)
147             val rxData = ByteArray(2)
148             spi?.transfer(txData, rxData)
149             rxData[0] == RESPONSE_OK
150         } catch (e: Exception) {
151             logger.warn("SPI connection test failed: ${e.message}")
152             false
153         }
154     }
155
156     fun isConnected(): Boolean = connected && enabled

```

157 }

Listing 16: Esp32SpiAdapter.kt

10.6.3 ESP32 TCP Adapter

```

1 package com.star.gateway.hardware
2
3 import com.fasterxml.jackson.databind.ObjectMapper
4 import com.star.gateway.model.CommandResult
5 import kotlinx.coroutines.Dispatchers
6 import kotlinx.coroutines.withContext
7 import kotlinx.coroutines.withTimeoutOrNull
8 import org.slf4j.LoggerFactory
9 import org.springframework.beans.factory.annotation.Value
10 import org.springframework.stereotype.Component
11 import java.io.BufferedReader
12 import java.io.InputStreamReader
13 import java.io.PrintWriter
14 import java.net.InetSocketAddress
15 import java.net.Socket
16
17 @Component
18 class Esp32TcpAdapter(
19     @Value("\${star.esp32.tcp.enabled:true}")
20     private val enabled: Boolean,
21
22     @Value("\${star.esp32.tcp.host:192.168.4.2}")
23     private val host: String,
24
25     @Value("\${star.esp32.tcp.port:5000}")
26     private val port: Int,
27
28     @Value("\${star.esp32.tcp.timeout-ms:100}")
29     private val timeoutMs: Long,
30
31     @Value("\${star.esp32.tcp.retry-attempts:3}")
32     private val retryAttempts: Int,
33
34     private val objectMapper: ObjectMapper
35 ) {
36     private val logger = LoggerFactory.getLogger(javaClass)
37
38     data class TcpCommand(
39         val command: String,
40         val params: Map<String, Any> = emptyMap(),
41         val timestamp: Long = System.currentTimeMillis()
42     )
43
44     data class TcpResponse(
45         val success: Boolean,
46         val message: String,
47         val data: Map<String, Any>? = null

```

```

48     )
49
50     /**
51      * Send velocity command via TCP.
52      */
53     suspend fun sendVelocityCommand(left: Float, right: Float):
54         CommandResult {
55         val command = TcpCommand(
56             command = "velocity",
57             params = mapOf(
58                 "left" to left,
59                 "right" to right
60             )
61         )
62         return sendCommand(command)
63     }
64
65     /**
66      * Send emergency stop via TCP.
67      */
68     suspend fun emergencyStop(): CommandResult {
69         val command = TcpCommand(command = "emergency_stop")
70         return sendCommand(command)
71     }
72
73     /**
74      * Generic command sender with retry logic.
75      */
76     private suspend fun sendCommand(command: TcpCommand): CommandResult {
77         if (!enabled) {
78             return CommandResult.failure("TCP adapter disabled")
79         }
80
81         var lastException: Exception? = null
82
83         repeat(retryAttempts) { attempt ->
84             try {
85                 return withContext(Dispatchers.IO) {
86                     withTimeoutOrNull(timeoutMs) {
87                         executeCommand(command)
88                     } ?: CommandResult.failure("TCP timeout after
89                         ${timeoutMs}ms")
90                 }
91             } catch (e: Exception) {
92                 lastException = e
93                 logger.warn("TCP attempt ${attempt + 1}/${retryAttempts}
94                     failed: ${e.message}")
95             }
96         }
97
98         return CommandResult.failure(
99             "TCP failed after $retryAttempts attempts:
100                 ${lastException?.message}"
101         )

```

```

98     }
99
100     private fun executeCommand(command: TcpCommand): CommandResult {
101         Socket().use { socket ->
102             socket.connect(InetSocketAddress(host, port),
103                 timeoutMs.toInt())
104             socket.soTimeout = timeoutMs.toInt()
105
106             val writer = PrintWriter(socket.getOutputStream(), true)
107             val reader =
108                 BufferedReader(InputStreamReader(socket.getInputStream()))
109
110             // Send JSON command
111             val json = objectMapper.writeValueAsString(command)
112             writer.println(json)
113
114             // Read response
115             val responseLine = reader.readLine()
116             val response = objectMapper.readValue(responseLine,
117                 TcpResponse::class.java)
118
119             return if (response.success) {
120                 CommandResult.success(response.message)
121             } else {
122                 CommandResult.failure(response.message)
123             }
124         }
125     }
126
127     /**
128      * Check TCP connectivity.
129      */
130     fun isConnected(): Boolean {
131         if (!enabled) return false
132
133         return try {
134             Socket().use { socket ->
135                 socket.connect(InetSocketAddress(host, port),
136                     timeoutMs.toInt())
137                 true
138             }
139         } catch (e: Exception) {
140             false
141         }
142     }
143 }

```

Listing 17: Esp32TcpAdapter.kt

10.6.4 GraphQL Controller

```

1 package com.star.gateway.graphql
2

```

```

3 import com.star.gateway.model.*
4 import com.star.gateway.service.TelemetryService
5 import com.star.gateway.service.MotorControlService
6 import kotlinx.coroutines.flow.Flow
7 import org.springframework.graphql.data.method.annotation.Argument
8 import org.springframework.graphql.data.method.annotation.MutationMapping
9 import org.springframework.graphql.data.method.annotation.QueryMapping
10 import
    org.springframework.graphql.data.method.annotation.SubscriptionMapping
11 import org.springframework.stereotype.Controller
12 import reactor.core.publisher.Flux
13
14 @Controller
15 class TelemetryController(
16     private val telemetryService: TelemetryService,
17     private val motorControlService: MotorControlService
18 ) {
19     // ===== QUERIES =====
20
21     @QueryMapping
22     suspend fun telemetry(): Telemetry {
23         return telemetryService.getCurrentTelemetry()
24     }
25
26     @QueryMapping
27     suspend fun systemStatus(): SystemStatus {
28         return telemetryService.getSystemStatus()
29     }
30
31     @QueryMapping
32     suspend fun telemetryHistory(
33         @Argument limit: Int?
34     ): List<TelemetrySnapshot> {
35         return telemetryService.getHistory(limit ?: 100)
36     }
37
38     // ===== MUTATIONS =====
39
40     @MutationMapping
41     suspend fun emergencyStop(): Boolean {
42         val result = motorControlService.emergencyStop()
43         return result.success
44     }
45
46     @MutationMapping
47     suspend fun setMode(@Argument mode: RobotMode): Boolean {
48         return telemetryService.setMode(mode)
49     }
50
51     // ===== SUBSCRIPTIONS =====
52
53     @SubscriptionMapping
54     fun telemetryStream(): Flux<Telemetry> {
55         return telemetryService.telemetryFlow()

```

```

56         .asFlux()
57     }
58
59     @SubscriptionMapping
60     fun eventLog(): Flux<Event> {
61         return telemetryService.eventFlow()
62             .asFlux()
63     }
64 }
65
66 // Extension function to convert Flow to Flux
67 private fun <T> Flow<T>.asFlux(): Flux<T> {
68     return kotlinox.coroutines.reactor.asFlux(this)
69 }

```

Listing 18: TelemetryController.kt

10.6.5 gRPC Service

```

1 package com.star.gateway.grpc
2
3 import com.star.gateway.proto.*
4 import com.star.gateway.service.MotorControlService
5 import io.grpc.stub.StreamObserver
6 import kotlinox.coroutines.flow.Flow
7 import kotlinox.coroutines.flow.flow
8 import kotlinox.coroutines.runBlocking
9 import net.devh.boot.grpc.server.service.GrpcService
10 import org.slf4j.LoggerFactory
11
12 @GrpcService
13 class MotorControlGrpcService(
14     private val motorControlService: MotorControlService
15 ) : MotorControlGrpcKt.MotorControlCoroutineImplBase() {
16
17     private val logger = LoggerFactory.getLogger(javaClass)
18
19     /**
20      * Handle velocity command via gRPC.
21      */
22     override suspend fun setVelocity(request: VelocityCommand):
23         CommandResponse {
24         val startTime = System.nanoTime()
25
26         logger.debug("gRPC SetVelocity: left=${request.leftVelocity},
27             right=${request.rightVelocity}")
28
29         val result = motorControlService.setVelocity(
30             left = request.leftVelocity,
31             right = request.rightVelocity
32         )
33
34         val latencyUs = (System.nanoTime() - startTime) / 1000

```

```
33
34     return CommandResponse.newBuilder()
35         .setSuccess(result.success)
36         .setMessage(result.message)
37         .setLatencyUs(latencyUs)
38         .build()
39 }
40
41 /**
42  * Handle emergency stop via gRPC.
43  */
44 override suspend fun emergencyStop(request: EmptyRequest):
45     CommandResponse {
46     logger.warn("gRPC EmergencyStop received")
47
48     val result = motorControlService.emergencyStop()
49
50     return CommandResponse.newBuilder()
51         .setSuccess(result.success)
52         .setMessage(result.message)
53         .build()
54 }
55
56 /**
57  * Stream encoder data to client.
58  */
59 override fun streamEncoderData(request: EmptyRequest):
60     Flow<EncoderData> = flow {
61     // Emit encoder data at 100Hz
62     while (true) {
63         val encoders = motorControlService.getEncoderData()
64
65         encoders.forEach { (motorId, data) ->
66             emit(
67                 EncoderData.newBuilder()
68                     .setMotorId(motorId)
69                     .setTicks(data.ticks)
70                     .setVelocity(data.velocity)
71                     .setTimestamp(System.currentTimeMillis())
72                     .build()
73             )
74         }
75
76         kotlinx.coroutines.delay(10) // 100Hz
77     }
78 }
```

Listing 19: MotorControlGrpcService.kt

11 PWA Frontend Implementation

11.1 Package Configuration

The PWA builds upon the existing UI-Demo codebase with additional dependencies:

```

1 {
2   "name": "star-ui",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "dev": "vite",
7     "build": "vue-tsc && vite build",
8     "preview": "vite preview",
9     "type-check": "vue-tsc --noEmit",
10    "generate:proto": "grpc_tools_node_protoc
        --js_out=import_style=commonjs,binary:./src/grpc/generated
        --grpc-web_out=import_style=typescript,mode=grpcwebtext:./src/grpc/generated
        -I ../proto ../proto/star/*.proto"
11  },
12  "dependencies": {
13    // Vue 3 Core
14    "vue": "^3.5.13",
15    "vue-router": "^4.5.0",
16    "pinia": "^2.3.0",
17
18    // UI Libraries
19    "chart.js": "^4.4.7",
20    "vue-chartjs": "^5.3.2",
21    "@vueuse/core": "^12.0.0",
22    "lucide-vue-next": "^0.460.0",
23    "clsx": "^2.1.1",
24    "tailwind-merge": "^3.4.0",
25
26    // GraphQL (Vue Apollo)
27    "@vue/apollo-composable": "^4.3.0",
28    "@apollo/client": "^3.12.0",
29    "graphql": "^16.10.0",
30    "graphql-ws": "^5.16.0",
31    "apollo3-cache-persist": "^0.15.0",
32
33    // gRPC-Web
34    "@improbable-eng/grpc-web": "^0.15.0",
35    "google-protobuf": "^3.21.4",
36
37    // Offline storage
38    "idb": "^8.0.1",
39
40    // Service Worker
41    "workbox-core": "^7.3.0",
42    "workbox-precaching": "^7.3.0",
43    "workbox-routing": "^7.3.0",
44    "workbox-strategies": "^7.3.0"
45  },

```

```

46  "devDependencies": {
47    // TypeScript
48    "typescript": "~5.7.0",
49    "vue-tsc": "^2.2.0",
50    "@types/node": "^22.10.0",
51
52    // Vite + Vue
53    "@vitejs/plugin-vue": "^5.2.1",
54    "vite": "^6.0.0",
55
56    // Styling
57    "tailwindcss": "^4.1.17",
58    "autoprefixer": "^10.4.20",
59
60    // PWA plugin
61    "vite-plugin-pwa": "^0.21.1",
62
63    // Proto compilation
64    "grpc_tools_node_protoc_ts": "^5.3.3",
65    "grpc-tools": "^1.12.4"
66  }
67 }

```

Listing 20: package.json (Vue 3 + TypeScript)

11.2 Vite PWA Configuration

```

1  import { defineConfig } from 'vite'
2  import vue from '@vitejs/plugin-vue'
3  import { VitePWA } from 'vite-plugin-pwa'
4
5  export default defineConfig({
6    plugins: [
7      vue(),
8      VitePWA({
9        registerType: 'autoUpdate',
10       includeAssets: [
11         'favicon.ico',
12         'robots.txt',
13         'apple-touch-icon.png'
14       ],
15       manifest: {
16         name: 'STAR Robot Controller',
17         short_name: 'STAR',
18         description: 'Control interface for STAR LiDAR SLAM robot',
19         theme_color: '#500000', // Texas A&M maroon
20         background_color: '#1a1a1a',
21         display: 'standalone',
22         orientation: 'any',
23         start_url: '/',
24         scope: '/',
25         icons: [
26

```

```

27         src: 'pwa-192x192.png',
28         sizes: '192x192',
29         type: 'image/png'
30     },
31     {
32         src: 'pwa-512x512.png',
33         sizes: '512x512',
34         type: 'image/png',
35         purpose: 'any maskable'
36     }
37 ]
38 },
39 workbox: {
40     // Precache static assets
41     globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
42
43     // Runtime caching strategies
44     runtimeCaching: [
45         {
46             // GraphQL API - Network First with fallback
47             urlPattern: /^https?:\/\/\/star-robot\.local(:8080)?\/graphql/,
48             handler: 'NetworkFirst',
49             options: {
50                 cacheName: 'graphql-cache',
51                 expiration: {
52                     maxEntries: 100,
53                     maxAgeSeconds: 60 * 60 // 1 hour
54                 },
55                 networkTimeoutSeconds: 3,
56                 cacheableResponse: {
57                     statuses: [0, 200]
58                 }
59             }
60         },
61         {
62             // Static assets from gateway
63             urlPattern: /^https?:\/\/\/star-robot\.local(:8080)?\/static/,
64             handler: 'CacheFirst',
65             options: {
66                 cacheName: 'static-assets',
67                 expiration: {
68                     maxEntries: 60,
69                     maxAgeSeconds: 60 * 60 * 24 * 30 // 30 days
70                 }
71             }
72         }
73     ]
74 },
75 devOptions: {
76     enabled: true // Enable PWA in dev mode for testing
77 }
78 })
79 ],
80 server: {

```

```

81   port: 3000,
82   proxy: {
83     '/graphql': {
84       target: 'http://localhost:8080',
85       ws: true // WebSocket proxy for subscriptions
86     },
87     '/grpc': {
88       target: 'http://localhost:8090'
89     }
90   }
91 }
92 })

```

Listing 21: vite.config.ts

11.3 Apollo Client with Offline Support (Vue 3)

```

1  import {
2    ApolloClient,
3    InMemoryCache,
4    HttpLink,
5    split,
6    ApolloLink,
7    type NormalizedCacheObject
8  } from '@apollo/client/core'
9  import { GraphQLWsLink } from '@apollo/client/link/subscriptions'
10 import { getMainDefinition } from '@apollo/client/utilities'
11 import { createClient } from 'graphql-ws'
12 import { persistCache, LocalStorageWrapper } from 'apollo3-cache-persist'
13 import { onError } from '@apollo/client/link/error'
14 import { provideApolloClient } from '@vue/apollo-composable'
15
16 // Gateway URL - defaults to mDNS hostname
17 const GATEWAY_URL = import.meta.env.VITE_GATEWAY_URL
18   || 'http://star-robot.local:8080';
19
20 // HTTP link for queries and mutations
21 const httpLink = new HttpLink({
22   uri: `${GATEWAY_URL}/graphql`,
23   credentials: 'include'
24 });
25
26 // WebSocket link for subscriptions
27 const wsLink = new GraphQLWsLink(createClient({
28   url: `ws://${GATEWAY_URL.replace(/^https?:\/\//, '')}/graphql-ws`,
29   connectionParams: () => ({
30     // Add auth tokens here if needed
31     timestamp: Date.now()
32   }),
33   retryAttempts: Infinity,
34   shouldRetry: () => true,
35   on: {
36     connected: () => console.log('WebSocket connected'),

```

```

37     closed: () => console.log('WebSocket closed'),
38     error: (err) => console.error('WebSocket error:', err)
39   }
40 }));
41
42 // Error handling link
43 const errorLink = onError(({ graphqlErrors, networkError, operation }) =>
44   {
45     if (graphqlErrors) {
46       graphqlErrors.forEach(({ message, locations, path }) => {
47         console.error(
48           '[GraphQL error]: Message: ${message}, ' +
49           'Location: ${locations}, Path: ${path}'
50         );
51       });
52     }
53     if (networkError) {
54       console.error('[Network error]: ${networkError}');
55       // Could dispatch to offline queue here
56     }
57   });
58
59 // Split link based on operation type
60 const splitLink = split(
61   ({ query }) => {
62     const definition = getMainDefinition(query);
63     return (
64       definition.kind === 'OperationDefinition' &&
65       definition.operation === 'subscription'
66     );
67   },
68   wsLink,
69   httpLink
70 );
71
72 // Create cache with custom type policies
73 const cache = new InMemoryCache({
74   typePolicies: {
75     Query: {
76       fields: {
77         telemetryHistory: {
78           // Merge incoming with existing, keep last 1000
79           merge(existing = [], incoming) {
80             return [...incoming].slice(-1000);
81           }
82         }
83       }
84     },
85     Telemetry: {
86       // Use timestamp as unique identifier
87       keyFields: ['timestamp']
88     },
89     Waypoint: {

```

```

90     keyFields: ['id']
91   }
92 }
93 });
94
95 // Initialize cache persistence
96 let apolloClient;
97
98 export async function initializeApollo() {
99   // Persist cache to localStorage
100   await persistCache({
101     cache,
102     storage: new LocalStorageWrapper(window.localStorage),
103     maxSize: 1048576 * 10, // 10MB
104     debug: import.meta.env.DEV
105   });
106
107   apolloClient = new ApolloClient({
108     link: ApolloLink.from([errorLink, splitLink]),
109     cache,
110     defaultOptions: {
111       watchQuery: {
112         fetchPolicy: 'cache-and-network',
113         errorPolicy: 'all'
114       },
115       query: {
116         fetchPolicy: 'cache-first',
117         errorPolicy: 'all'
118       },
119       mutate: {
120         errorPolicy: 'all'
121       }
122     }
123   });
124
125   return apolloClient;
126 }
127
128 export function getApolloClient() {
129   if (!apolloClient) {
130     throw new Error('Apollo Client not initialized. Call
131       initializeApollo() first.');
```

Listing 22: src/graphql/client.ts

11.4 GraphQL Operations

```

1 import { gql } from '@apollo/client';
2
3 // ===== QUERIES =====
```

```
4
5 export const TELEMETRY_QUERY = gql`
6   query GetTelemetry {
7     telemetry {
8       imu {
9         pitch
10        roll
11        yaw
12        velocity
13        accelerationX
14        accelerationY
15        accelerationZ
16      }
17      gps {
18        latitude
19        longitude
20        altitude
21      }
22      battery
23      wifiSignal
24      cpuUsage
25      temperature
26      motorLoad
27      timestamp
28    }
29  }
30 `;
31
32 export const SYSTEM_STATUS_QUERY = gql`
33   query GetSystemStatus {
34     systemStatus {
35       connectionStatus
36       mode
37       esp32Connected
38       lidarConnected
39       rosConnected
40     }
41   }
42 `;
43
44 export const NAVIGATION_STATE_QUERY = gql`
45   query GetNavigationState {
46     navigationState {
47       currentPosition {
48         x
49         y
50         heading
51       }
52       path {
53         x
54         y
55         heading
56       }
57       waypoints {
```

```

58         id
59         x
60         y
61         label
62     }
63     isNavigating
64 }
65 }
66 ';
67
68 export const TELEMETRY_HISTORY_QUERY = gql`
69     query GetTelemetryHistory($limit: Int) {
70         telemetryHistory(limit: $limit) {
71             imu {
72                 pitch
73                 roll
74                 yaw
75                 velocity
76             }
77             battery
78             timestamp
79         }
80     }
81 `;
82
83 // ===== MUTATIONS =====
84
85 export const EMERGENCY_STOP_MUTATION = gql`
86     mutation EmergencyStop {
87         emergencyStop
88     }
89 `;
90
91 export const SET_MODE_MUTATION = gql`
92     mutation SetMode($mode: RobotMode!) {
93         setMode(mode: $mode)
94     }
95 `;
96
97 export const SET_SETTINGS_MUTATION = gql`
98     mutation SetSettings($input: SettingsInput!) {
99         setSettings(input: $input) {
100             maxSpeed
101             safetyLockEnabled
102             autonomousEnabled
103             lidarEnabled
104         }
105     }
106 `;
107
108 export const ADD_WAYPOINT_MUTATION = gql`
109     mutation AddWaypoint($x: Float!, $y: Float!) {
110         addWaypoint(x: $x, y: $y) {
111             id

```

```
112     x
113     y
114     label
115   }
116 }
117 ‘;
118
119 // ===== SUBSCRIPTIONS =====
120
121 export const TELEMETRY_SUBSCRIPTION = gql‘
122   subscription TelemetryStream {
123     telemetryStream {
124       imu {
125         pitch
126         roll
127         yaw
128         velocity
129       }
130       battery
131       wifiSignal
132       cpuUsage
133       temperature
134       timestamp
135     }
136   }
137 ‘;
138
139 export const LIDAR_SCAN_SUBSCRIPTION = gql‘
140   subscription LidarScan {
141     lidarScan {
142       points {
143         angle
144         distance
145         intensity
146       }
147       timestamp
148     }
149   }
150 ‘;
151
152 export const EVENT_LOG_SUBSCRIPTION = gql‘
153   subscription EventLog {
154     eventLog {
155       timestamp
156       message
157       type
158     }
159   }
160 ‘;
```

Listing 23: src/graphql/operations.js

11.5 gRPC-Web Client

```

1 import { grpc } from '@improbable-eng/grpc-web';
2 import {
3   MotorControlClient
4 } from '../generated/motor_control_pb_service';
5 import {
6   VelocityCommand,
7   EmptyRequest,
8   MotorPowerCommand
9 } from '../generated/motor_control_pb';
10
11 // gRPC endpoint (through Envoy proxy)
12 const GRPC_URL = import.meta.env.VITE_GRPC_URL
13   || 'http://star-robot.local:8090';
14
15 // Create gRPC-Web client
16 const client = new MotorControlClient(GRPC_URL);
17
18 /**
19  * Send velocity command via gRPC.
20  * @param {number} leftVelocity - Left wheel velocity in m/s
21  * @param {number} rightVelocity - Right wheel velocity in m/s
22  * @returns {Promise<{success: boolean, message: string, latencyUs:
23    number}>}
24  */
25 export function setVelocity(leftVelocity, rightVelocity) {
26   return new Promise((resolve, reject) => {
27     const request = new VelocityCommand();
28     request.setLeftVelocity(leftVelocity);
29     request.setRightVelocity(rightVelocity);
30     request.setTimestamp(Date.now());
31
32     client.setVelocity(request, {}, (err, response) => {
33       if (err) {
34         console.error('gRPC setVelocity error:', err);
35         reject(err);
36       } else {
37         resolve({
38           success: response.getSuccess(),
39           message: response.getMessage(),
40           latencyUs: response.getLatencyUs()
41         });
42       }
43     });
44   });
45
46   /**
47    * Send emergency stop command via gRPC.
48    * @returns {Promise<{success: boolean, message: string}>}
49    */
50   export function emergencyStop() {
51     return new Promise((resolve, reject) => {

```

```

52     const request = new EmptyRequest();
53
54     client.emergencyStop(request, {}, (err, response) => {
55         if (err) {
56             console.error('gRPC emergencyStop error:', err);
57             reject(err);
58         } else {
59             resolve({
60                 success: response.getSuccess(),
61                 message: response.getMessage()
62             });
63         }
64     });
65 });
66 }
67
68 /**
69  * Set individual motor power via gRPC.
70  * @param {number} motorId - Motor index (0-3)
71  * @param {number} power - Power level (-1.0 to 1.0)
72  * @returns {Promise<{success: boolean, message: string}>}
73  */
74 export function setMotorPower(motorId, power) {
75     return new Promise((resolve, reject) => {
76         const request = new MotorPowerCommand();
77         request.setMotorId(motorId);
78         request.setPower(power);
79
80         client.setMotorPower(request, {}, (err, response) => {
81             if (err) {
82                 reject(err);
83             } else {
84                 resolve({
85                     success: response.getSuccess(),
86                     message: response.getMessage()
87                 });
88             }
89         });
90     });
91 }
92
93 /**
94  * Stream encoder data from all motors.
95  * @param {Function} onData - Callback for each encoder data packet
96  * @param {Function} onError - Callback for errors
97  * @param {Function} onEnd - Callback when stream ends
98  * @returns {Function} Cancel function to stop the stream
99  */
100 export function streamEncoderData(onData, onError, onEnd) {
101     const request = new EmptyRequest();
102     const stream = client.streamEncoderData(request);
103
104     stream.on('data', (response) => {
105         onData({

```

```

106     motorId: response.getMotorId(),
107     ticks: response.getTicks(),
108     velocity: response.getVelocity(),
109     timestamp: response.getTimestamp()
110   });
111 });
112
113 stream.on('error', (err) => {
114   console.error('Encoder stream error:', err);
115   onError(err);
116 });
117
118 stream.on('end', () => {
119   console.log('Encoder stream ended');
120   onEnd();
121 });
122
123 // Return cancel function
124 return () => {
125   stream.cancel();
126 };
127 }

```

Listing 24: src/grpc/client.js

11.6 Robot Connection Composable (Vue 3 - Replaces useMockData)

```

1 import { ref, computed, onMounted, onUnmounted, watch } from 'vue'
2 import { useQuery, useSubscription, useMutation } from
   '@vue/apollo-composable'
3 import {
4   TELEMETRY_QUERY,
5   SYSTEM_STATUS_QUERY,
6   NAVIGATION_STATE_QUERY,
7   TELEMETRY_SUBSCRIPTION,
8   LIDAR_SCAN_SUBSCRIPTION,
9   EVENT_LOG_SUBSCRIPTION,
10  EMERGENCY_STOP_MUTATION,
11  SET_MODE_MUTATION
12 } from '@graphql/operations'
13 import * as grpcClient from '@grpc/client'
14 import {
15   saveTelemetryToIndexedDB,
16   getTelemetryFromIndexedDB,
17   saveNavigationState,
18   getNavigationState as getStoredNavState
19 } from '@storage/indexedDB'
20 import type { Telemetry, SystemStatus, NavigationState, EncoderData }
   from '@types'
21
22 /**
23  * Main composable for robot connection and control.
24  * Replaces useMockData with real API integration.

```

```

25  */
26  export function useRobotConnection() {
27    // Connection state (Vue 3 refs)
28    const isOnline = ref(navigator.onLine)
29    const lastSyncTime = ref<Date | null>(null)
30    const connectionQuality = ref<'good' | 'fair' | 'poor'>('good')
31
32    // Encoder stream reference
33    let encoderStreamCancel: (() => void) | null = null
34    const encoderData = ref<Record<number, EncoderData>>({})
35
36    // ===== GraphQL Queries (Vue Apollo) =====
37
38    const {
39      result: telemetryResult,
40      loading: telemetryLoading,
41      error: telemetryError
42    } = useQuery<TELEMETRY_QUERY>(), () => ({
43      pollInterval: isOnline.value ? 0 : 5000,
44      fetchPolicy: 'cache-and-network'
45    })
46
47    const { result: statusResult } = useQuery<SYSTEM_STATUS_QUERY>(), {
48      pollInterval: 1000
49    }
50
51    const { result: navResult } = useQuery<NAVIGATION_STATE_QUERY>(), {
52      pollInterval: 500
53    }
54
55    // ===== GraphQL Subscriptions =====
56
57    const { result: liveTelemetry } = useSubscription<
58      TELEMETRY_SUBSCRIPTION,
59      null,
60      () => ({ enabled: isOnline.value })
61    >()
62
63    const { result: lidarResult } = useSubscription<
64      LIDAR_SCAN_SUBSCRIPTION,
65      null,
66      () => ({ enabled: isOnline.value })
67    >()
68
69    const { result: eventResult } = useSubscription<
70      EVENT_LOG_SUBSCRIPTION,
71      null,
72      () => ({ enabled: isOnline.value })
73    >()
74
75    // Watch for telemetry updates to persist to IndexedDB
76    watch<liveTelemetry>(), (newData) => {
77      if (newData?.telemetryStream) {
78        saveTelemetryToIndexedDB(newData.telemetryStream)

```

```

79     lastSyncTime.value = new Date()
80   }
81 })
82
83 // ===== GraphQL Mutations =====
84
85 const { mutate: emergencyStopMutation } =
86   useMutation(EMERGENCY_STOP_MUTATION)
87 const { mutate: setModeMutation } = useMutation(SET_MODE_MUTATION)
88
89 // ===== Network Status (Vue lifecycle) =====
90
91 onMounted(() => {
92   const handleOnline = () => {
93     isOnline.value = true
94     console.log('Network: Online')
95   }
96   const handleOffline = () => {
97     isOnline.value = false
98     console.log('Network: Offline')
99   }
100   window.addEventListener('online', handleOnline)
101   window.addEventListener('offline', handleOffline)
102 })
103
104 onUnmounted(() => {
105   window.removeEventListener('online', () => {})
106   window.removeEventListener('offline', () => {})
107 })
108
109 // ===== Encoder Stream (gRPC) =====
110
111 watch(isOnline, (online) => {
112   if (online && !encoderStreamCancel) {
113     encoderStreamCancel = grpcClient.streamEncoderData(
114       (data: EncoderData) => {
115         encoderData.value = {
116           ...encoderData.value,
117           [data.motorId]: data
118         }
119       },
120       (err: Error) => {
121         console.error('Encoder stream error:', err)
122         encoderStreamCancel = null
123       },
124       () => {
125         encoderStreamCancel = null
126       }
127     )
128   }
129 }, { immediate: true })
130
131 onUnmounted(() => {

```

```

132     if (encoderStreamCancel) {
133         encoderStreamCancel()
134         encoderStreamCancel = null
135     }
136 })
137
138 // ===== Motor Control (gRPC) =====
139
140 async function sendMotorCommand(left: number, right: number) {
141     if (!isOnline.value) {
142         throw new Error('Cannot send motor commands while offline')
143     }
144     return grpcClient.setVelocity(left, right)
145 }
146
147 async function emergencyStop() {
148     // Try both gRPC and GraphQL for redundancy
149     const results = await Promise.allSettled([
150         grpcClient.emergencyStop(),
151         emergencyStopMutation()
152     ])
153
154     const anySuccess = results.some(
155         r => r.status === 'fulfilled' && (r.value as any)?.success !== false
156     )
157
158     return { success: anySuccess }
159 }
160
161 async function setMode(mode: string) {
162     const result = await setModeMutation({ mode })
163     return result?.data?.setMode
164 }
165
166 // ===== Computed Properties =====
167
168 const telemetry = computed(() =>
169     liveTelemetry.value?.telemetryStream
170     || telemetryResult.value?.telemetry
171 )
172
173 const lidarScan = computed(() => lidarResult.value?.lidarScan)
174 const systemStatus = computed(() => statusResult.value?.systemStatus)
175 const navigationState = computed(() => navResult.value?.navigationState)
176 const latestEvent = computed(() => eventResult.value?.eventLog)
177
178 // ===== Return Values =====
179
180 return {
181     // Connection state
182     isOnline,
183     lastSyncTime,
184     connectionQuality,
185

```

```

186 // Telemetry
187 telemetry,
188 telemetryLoading,
189 telemetryError,
190
191 // LiDAR
192 lidarScan,
193
194 // System status
195 systemStatus,
196
197 // Navigation
198 navigationState,
199
200 // Events
201 latestEvent,
202
203 // Encoder data
204 encoderData,
205
206 // Motor control
207 sendMotorCommand,
208 emergencyStop,
209 setMode
210 }
211 }

```

Listing 25: src/composables/useRobotConnection.ts

11.7 IndexedDB Offline Storage

```

1 import { openDB } from 'idb';
2
3 const DB_NAME = 'star-robot-db';
4 const DB_VERSION = 1;
5
6 let dbPromise;
7
8 /**
9  * Get or create the IndexedDB database.
10  */
11 async function getDB() {
12   if (!dbPromise) {
13     dbPromise = openDB(DB_NAME, DB_VERSION, {
14       upgrade(db, oldVersion, newVersion, transaction) {
15         // Telemetry history store
16         if (!db.objectStoreNames.contains('telemetry')) {
17           const store = db.createObjectStore('telemetry', {
18             keyPath: 'id',
19             autoIncrement: true
20           });
21           store.createIndex('timestamp', 'timestamp');
22           store.createIndex('savedAt', 'savedAt');

```

```

23     }
24
25     // Settings store
26     if (!db.objectStoreNames.contains('settings')) {
27         db.createObjectStore('settings', { keyPath: 'key' });
28     }
29
30     // Navigation state store
31     if (!db.objectStoreNames.contains('navigation')) {
32         db.createObjectStore('navigation', { keyPath: 'key' });
33     }
34
35     // Event log store
36     if (!db.objectStoreNames.contains('events')) {
37         const store = db.createObjectStore('events', {
38             keyPath: 'id',
39             autoIncrement: true
40         });
41         store.createIndex('timestamp', 'timestamp');
42     }
43 }
44 });
45 }
46 return dbPromise;
47 }
48
49 /**
50  * Save telemetry snapshot to IndexedDB.
51  * Automatically prunes to keep only last 10,000 entries.
52  */
53 export async function saveTelemetryToIndexedDB(telemetry) {
54     const db = await getDB();
55
56     await db.add('telemetry', {
57         ...telemetry,
58         savedAt: Date.now()
59     });
60
61     // Prune old entries
62     const count = await db.count('telemetry');
63     if (count > 10000) {
64         const tx = db.transaction('telemetry', 'readwrite');
65         const store = tx.objectStore('telemetry');
66         const index = store.index('savedAt');
67
68         let cursor = await index.openCursor();
69         const toDelete = count - 10000;
70         let deleted = 0;
71
72         while (cursor && deleted < toDelete) {
73             await cursor.delete();
74             cursor = await cursor.continue();
75             deleted++;
76         }

```

```
77
78     await tx.done;
79   }
80 }
81
82 /**
83  * Retrieve telemetry history from IndexedDB.
84  * @param {number} limit - Maximum entries to return
85  * @returns {Promise<Array>} Telemetry entries, newest first
86  */
87 export async function getTelemetryFromIndexedDB(limit = 100) {
88   const db = await getDB();
89   const tx = db.transaction('telemetry', 'readonly');
90   const store = tx.objectStore('telemetry');
91   const index = store.index('timestamp');
92
93   const entries = [];
94   let cursor = await index.openCursor(null, 'prev');
95
96   while (cursor && entries.length < limit) {
97     entries.push(cursor.value);
98     cursor = await cursor.continue();
99   }
100
101   return entries;
102 }
103
104 /**
105  * Save user settings to IndexedDB.
106  */
107 export async function saveSettings(settings) {
108   const db = await getDB();
109   await db.put('settings', {
110     key: 'userSettings',
111     ...settings,
112     updatedAt: Date.now()
113   });
114 }
115
116 /**
117  * Retrieve user settings from IndexedDB.
118  */
119 export async function getSettings() {
120   const db = await getDB();
121   const result = await db.get('settings', 'userSettings');
122   return result || null;
123 }
124
125 /**
126  * Save navigation state to IndexedDB.
127  */
128 export async function saveNavigationState(state) {
129   const db = await getDB();
130   await db.put('navigation', {
```

```
131     key: 'currentState',
132     ...state,
133     savedAt: Date.now()
134   });
135 }
136
137 /**
138  * Retrieve navigation state from IndexedDB.
139  */
140 export async function getNavigationState() {
141   const db = await getDB();
142   const result = await db.get('navigation', 'currentState');
143   return result || null;
144 }
145
146 /**
147  * Save event to IndexedDB.
148  */
149 export async function saveEvent(event) {
150   const db = await getDB();
151   await db.add('events', {
152     ...event,
153     savedAt: Date.now()
154   });
155
156   // Keep only last 500 events
157   const count = await db.count('events');
158   if (count > 500) {
159     const tx = db.transaction('events', 'readwrite');
160     const store = tx.objectStore('events');
161     const index = store.index('timestamp');
162
163     let cursor = await index.openCursor();
164     const toDelete = count - 500;
165     let deleted = 0;
166
167     while (cursor && deleted < toDelete) {
168       await cursor.delete();
169       cursor = await cursor.continue();
170       deleted++;
171     }
172   }
173 }
174
175 /**
176  * Get recent events from IndexedDB.
177  */
178 export async function getEvents(limit = 50) {
179   const db = await getDB();
180   const tx = db.transaction('events', 'readonly');
181   const store = tx.objectStore('events');
182   const index = store.index('timestamp');
183
184   const entries = [];
```

```
185   let cursor = await index.openCursor(null, 'prev');
186
187   while (cursor && entries.length < limit) {
188     entries.push(cursor.value);
189     cursor = await cursor.continue();
190   }
191
192   return entries;
193 }
194
195 /**
196  * Clear all data from IndexedDB.
197  */
198 export async function clearAllData() {
199   const db = await getDB();
200
201   const tx = db.transaction(
202     ['telemetry', 'settings', 'navigation', 'events'],
203     'readwrite'
204   );
205
206   await Promise.all([
207     tx.objectStore('telemetry').clear(),
208     tx.objectStore('settings').clear(),
209     tx.objectStore('navigation').clear(),
210     tx.objectStore('events').clear()
211   ]);
212
213   await tx.done;
214 }
```

Listing 26: src/storage/indexedDB.js

12 ESP32 Firmware Updates

The existing STAR firmware requires additions for TCP communication as a fallback to SPI.

12.1 TCP Server Module

```

1  /**
2   * @file star_tcp_server.h
3   * @brief TCP server for command reception from RPi5
4   *
5   * Provides fallback communication when SPI is unavailable.
6   */
7
8  #ifndef STAR_TCP_SERVER_H
9  #define STAR_TCP_SERVER_H
10
11 #include "esp_err.h"
12
13 #ifdef __cplusplus
14 extern "C" {
15 #endif
16
17 #define TCP_SERVER_PORT 5000
18 #define TCP_MAX_CLIENTS 2
19 #define TCP_RX_BUFFER_SIZE 256
20 #define TCP_TX_BUFFER_SIZE 256
21
22 /**
23  * @brief Motor command callback type
24  */
25 typedef struct {
26     float left_velocity;
27     float right_velocity;
28     bool emergency_stop;
29     uint64_t timestamp;
30 } tcp_motor_command_t;
31
32 typedef void (*tcp_command_callback_t)(const tcp_motor_command_t*
33     command);
34
35 /**
36  * @brief TCP server configuration
37  */
38 typedef struct {
39     uint16_t port;
40     tcp_command_callback_t on_command;
41     void* user_context;
42 } tcp_server_config_t;
43
44 /**
45  * @brief TCP server handle
46  */
47 typedef struct tcp_server_handle* tcp_server_handle_t;

```

```

47
48 /**
49  * @brief Initialize and start TCP server
50  *
51  * @param config Server configuration
52  * @param out_handle Output handle
53  * @return esp_err_t ESP_OK on success
54  */
55 esp_err_t star_tcp_server_init(
56     const tcp_server_config_t* config,
57     tcp_server_handle_t* out_handle
58 );
59
60 /**
61  * @brief Stop and cleanup TCP server
62  */
63 esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle);
64
65 /**
66  * @brief Get number of connected clients
67  */
68 int star_tcp_server_get_client_count(tcp_server_handle_t handle);
69
70 #ifdef __cplusplus
71 }
72 #endif
73
74 #endif // STAR_TCP_SERVER_H

```

Listing 27: lib/star_module_tcp/include/star_tcp_server.h

```

1 /**
2  * @file star_tcp_server.c
3  * @brief TCP server implementation
4  */
5
6 #include "star_tcp_server.h"
7 #include "esp_log.h"
8 #include "esp_netif.h"
9 #include "lwip/sockets.h"
10 #include "cJSON.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13
14 static const char* TAG = "tcp_server";
15
16 struct tcp_server_handle {
17     int listen_socket;
18     tcp_command_callback_t callback;
19     void* user_context;
20     TaskHandle_t server_task;
21     bool running;
22     int client_count;
23 };

```

```

24
25 /**
26  * @brief Parse JSON command from client
27  */
28 static esp_err_t parse_json_command(
29     const char* json_str,
30     tcp_motor_command_t* out_cmd
31 ) {
32     cJSON* root = cJSON_Parse(json_str);
33     if (root == NULL) {
34         ESP_LOGE(TAG, "Failed to parse JSON");
35         return ESP_ERR_INVALID_ARG;
36     }
37
38     memset(out_cmd, 0, sizeof(tcp_motor_command_t));
39
40     cJSON* cmd = cJSON_GetObjectItem(root, "command");
41     if (cmd && cJSON_IsString(cmd)) {
42         if (strcmp(cmd->valuestring, "velocity") == 0) {
43             cJSON* params = cJSON_GetObjectItem(root, "params");
44             if (params) {
45                 cJSON* left = cJSON_GetObjectItem(params, "left");
46                 cJSON* right = cJSON_GetObjectItem(params, "right");
47
48                 if (left && cJSON_IsNumber(left)) {
49                     out_cmd->left_velocity = (float)left->valuedouble;
50                 }
51                 if (right && cJSON_IsNumber(right)) {
52                     out_cmd->right_velocity = (float)right->valuedouble;
53                 }
54             }
55         } else if (strcmp(cmd->valuestring, "emergency_stop") == 0) {
56             out_cmd->emergency_stop = true;
57         }
58     }
59
60     cJSON* ts = cJSON_GetObjectItem(root, "timestamp");
61     if (ts && cJSON_IsNumber(ts)) {
62         out_cmd->timestamp = (uint64_t)ts->valuedouble;
63     }
64
65     cJSON_Delete(root);
66     return ESP_OK;
67 }
68
69 /**
70  * @brief Build JSON response
71  */
72 static void build_json_response(
73     char* buffer,
74     size_t size,
75     bool success,
76     const char* message
77 ) {

```

```

78     snprintf(buffer, size,
79         "{ \"success\":%s, \"message\": \"%s\" }",
80         success ? "true" : "false",
81         message
82     );
83 }
84
85 /**
86  * @brief Handle single client connection
87  */
88 static void handle_client(
89     tcp_server_handle_t handle,
90     int client_socket
91 ) {
92     char rx_buffer[TCP_RX_BUFFER_SIZE];
93     char tx_buffer[TCP_TX_BUFFER_SIZE];
94
95     int len = recv(client_socket, rx_buffer, sizeof(rx_buffer) - 1, 0);
96
97     if (len > 0) {
98         rx_buffer[len] = '\0';
99         ESP_LOGD(TAG, "Received: %s", rx_buffer);
100
101         tcp_motor_command_t cmd;
102         esp_err_t err = parse_json_command(rx_buffer, &cmd);
103
104         if (err == ESP_OK && handle->callback) {
105             handle->callback(&cmd);
106             build_json_response(tx_buffer, sizeof(tx_buffer), true, "OK");
107         } else {
108             build_json_response(tx_buffer, sizeof(tx_buffer), false,
109                 "Parse error");
110         }
111
112         send(client_socket, tx_buffer, strlen(tx_buffer), 0);
113     }
114 }
115
116 /**
117  * @brief TCP server task
118  */
119 static void tcp_server_task(void* pvParameters) {
120     tcp_server_handle_t handle = (tcp_server_handle_t)pvParameters;
121
122     struct sockaddr_in server_addr = {
123         .sin_family = AF_INET,
124         .sin_port = htons(TCP_SERVER_PORT),
125         .sin_addr.s_addr = htonl(INADDR_ANY)
126     };
127
128     handle->listen_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
129     if (handle->listen_socket < 0) {
130         ESP_LOGE(TAG, "Failed to create socket");
131         vTaskDelete(NULL);
132     }

```

```

131     return;
132 }
133
134 int opt = 1;
135 setsockopt(handle->listen_socket, SOL_SOCKET, SO_REUSEADDR, &opt,
136           sizeof(opt));
137
138 if (bind(handle->listen_socket, (struct sockaddr*)&server_addr,
139       sizeof(server_addr)) < 0) {
140     ESP_LOGE(TAG, "Failed to bind socket");
141     close(handle->listen_socket);
142     vTaskDelete(NULL);
143     return;
144 }
145
146 if (listen(handle->listen_socket, TCP_MAX_CLIENTS) < 0) {
147     ESP_LOGE(TAG, "Failed to listen");
148     close(handle->listen_socket);
149     vTaskDelete(NULL);
150     return;
151 }
152
153 ESP_LOGI(TAG, "TCP server listening on port %d", TCP_SERVER_PORT);
154
155 while (handle->running) {
156     struct sockaddr_in client_addr;
157     socklen_t addr_len = sizeof(client_addr);
158
159     int client_socket = accept(
160         handle->listen_socket,
161         (struct sockaddr*)&client_addr,
162         &addr_len
163     );
164
165     if (client_socket >= 0) {
166         handle->client_count++;
167         ESP_LOGI(TAG, "Client connected from %s",
168             inet_ntoa(client_addr.sin_addr));
169
170         handle_client(handle, client_socket);
171
172         close(client_socket);
173         handle->client_count--;
174     }
175 }
176
177 close(handle->listen_socket);
178 vTaskDelete(NULL);
179 }
180
181 esp_err_t star_tcp_server_init(
182     const tcp_server_config_t* config,
183     tcp_server_handle_t* out_handle
184 ) {

```

```

184     if (config == NULL || out_handle == NULL) {
185         return ESP_ERR_INVALID_ARG;
186     }
187
188     tcp_server_handle_t handle = calloc(1, sizeof(struct
189         tcp_server_handle));
189     if (handle == NULL) {
190         return ESP_ERR_NO_MEM;
191     }
192
193     handle->callback = config->on_command;
194     handle->user_context = config->user_context;
195     handle->running = true;
196     handle->client_count = 0;
197
198     BaseType_t ret = xTaskCreate(
199         tcp_server_task,
200         "tcp_server",
201         4096,
202         handle,
203         5,
204         &handle->server_task
205     );
206
207     if (ret != pdPASS) {
208         free(handle);
209         return ESP_FAIL;
210     }
211
212     *out_handle = handle;
213     return ESP_OK;
214 }
215
216 esp_err_t star_tcp_server_deinit(tcp_server_handle_t handle) {
217     if (handle == NULL) {
218         return ESP_ERR_INVALID_ARG;
219     }
220
221     handle->running = false;
222
223     // Wake up the accept() call
224     if (handle->listen_socket >= 0) {
225         shutdown(handle->listen_socket, SHUT_RDWR);
226     }
227
228     // Wait for task to finish
229     vTaskDelay(pdMS_TO_TICKS(100));
230
231     free(handle);
232     return ESP_OK;
233 }
234
235 int star_tcp_server_get_client_count(tcp_server_handle_t handle) {
236     return handle ? handle->client_count : 0;

```

237 }

Listing 28: lib/star_module_tcp/src/star_tcp_server.c

12.2 Main Application Integration

Add TCP server to the main application:

```

1  // Add to includes
2  #include "star_tcp_server.h"
3
4  // Add global handle
5  static tcp_server_handle_t g_tcp_server = NULL;
6
7  // Add command callback
8  static void on_tcp_command(const tcp_motor_command_t* cmd) {
9      if (cmd->emergency_stop) {
10         // Execute emergency stop
11         motor_control_emergency_stop();
12     } else {
13         // Set motor velocities
14         motor_control_set_velocity(cmd->left_velocity,
15                                   cmd->right_velocity);
16     }
17 }
18
19 // Add to app_main() after WiFi is connected
20 void app_main(void) {
21     // ... existing initialization ...
22
23     // Initialize TCP server for RPi5 communication
24     tcp_server_config_t tcp_config = {
25         .port = TCP_SERVER_PORT,
26         .on_command = on_tcp_command,
27         .user_context = NULL
28     };
29
30     esp_err_t err = star_tcp_server_init(&tcp_config, &g_tcp_server);
31     if (err == ESP_OK) {
32         ESP_LOGI(TAG, "TCP server started on port %d", TCP_SERVER_PORT);
33     } else {
34         ESP_LOGE(TAG, "Failed to start TCP server");
35     }
36
37     // ... rest of initialization ...
38 }
```

Listing 29: Diff for ESP32-Firmware/src/main.c

13 Process Flow Diagrams

This section illustrates the data flow through the STAR robot system for key operations.

13.1 Motor Command Flow

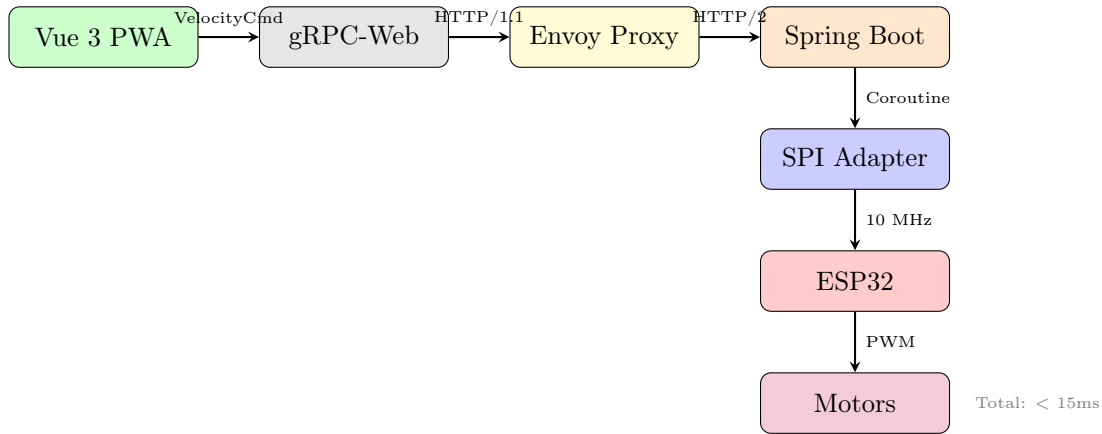


Figure 9: Motor Command Data Flow

13.2 SLAM Data Flow

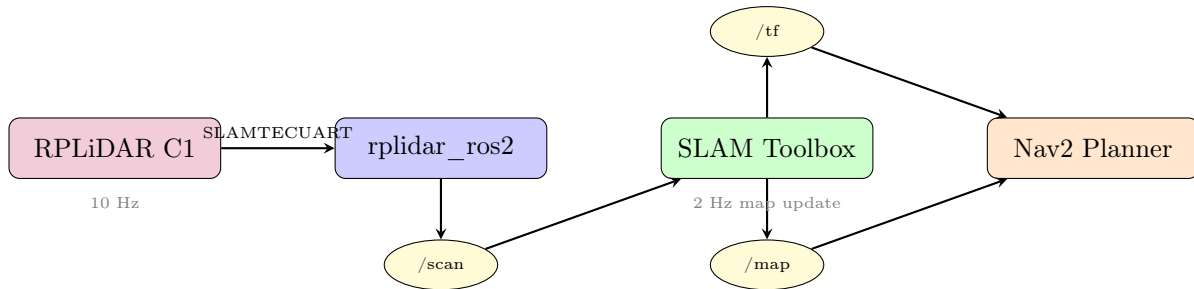


Figure 10: SLAM Data Flow (ROS 2 Topics)

13.3 Navigation Pipeline

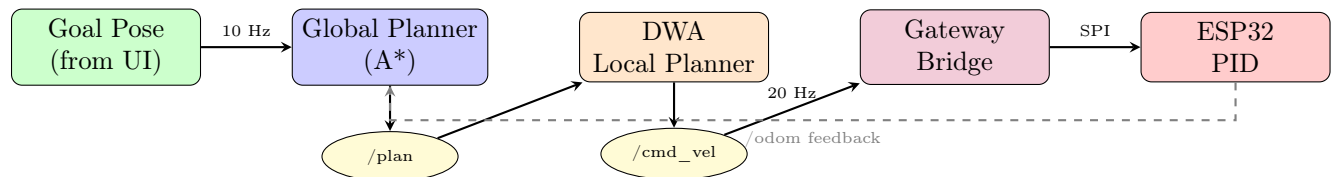


Figure 11: Navigation Pipeline Data Flow

13.4 Telemetry Aggregation Flow

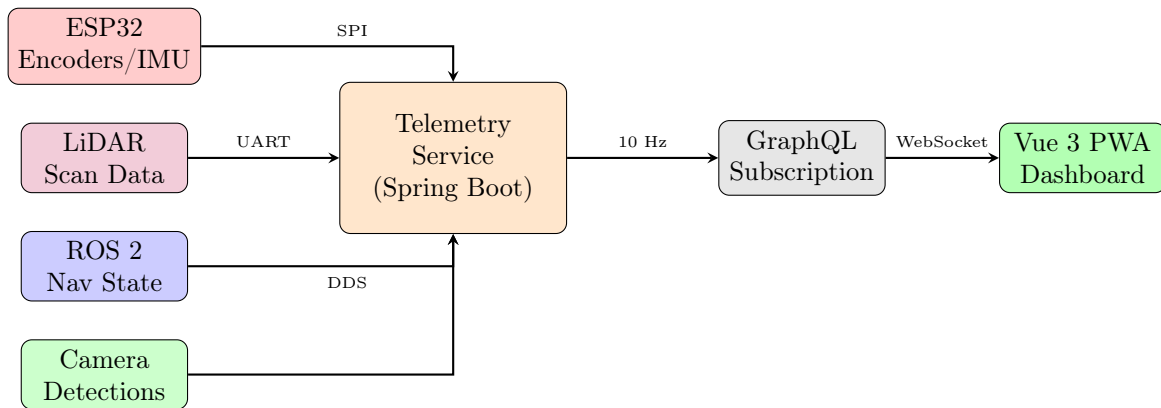


Figure 12: Telemetry Aggregation and Publishing

14 Deployment Configuration

14.1 Systemd Service

```

1 [Unit]
2 Description=STAR Robot Gateway
3 Documentation=https://github.com/locked-inc/star
4 After=network-online.target
5 Wants=network-online.target avahi-daemon.service
6
7 [Service]
8 Type=simple
9 User=star
10 Group=star
11 WorkingDirectory=/opt/star-gateway
12 ExecStart=/usr/bin/java \
13     -Xmx256m \
14     -Xms128m \
15     -XX:+UseG1GC \
16     -XX:MaxGCPauseMillis=50 \
17     -Djava.security.egd=file:/dev/./urandom \
18     -jar star-gateway.jar
19
20 # Restart policy
21 Restart=always
22 RestartSec=10
23 StartLimitInterval=300
24 StartLimitBurst=5
25
26 # Logging
27 StandardOutput=journal
28 StandardError=journal
29 SyslogIdentifier=star-gateway
30
31 # Environment
32 Environment="SPRING_PROFILES_ACTIVE=production"
33 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
34
35 # Security hardening
36 NoNewPrivileges=true
37 ProtectSystem=strict
38 ProtectHome=true
39 PrivateTmp=true
40 ReadWritePaths=/var/log/star-gateway /var/lib/star-gateway
41
42 # Resource limits
43 MemoryMax=512M
44 CPUQuota=80%
45
46 [Install]
47 WantedBy=multi-user.target

```

Listing 30: star-gateway/deployment/star-gateway.service

14.2 Avahi mDNS Service

```

1 <?xml version="1.0" standalone='no'?>
2 <!DOCTYPE service-group SYSTEM "avahi-service.dtd">
3 <service-group>
4   <name replace-wildcards="yes">STAR Robot Gateway on %h</name>
5
6   <!-- GraphQL HTTP endpoint -->
7   <service>
8     <type>_http._tcp</type>
9     <port>8080</port>
10    <txt-record>path=/graphql</txt-record>
11    <txt-record>api=graphql</txt-record>
12  </service>
13
14  <!-- GraphQL WebSocket endpoint -->
15  <service>
16    <type>_ws._tcp</type>
17    <port>8080</port>
18    <txt-record>path=/graphql-ws</txt-record>
19  </service>
20
21  <!-- gRPC endpoint (via Envoy) -->
22  <service>
23    <type>_grpc._tcp</type>
24    <port>8090</port>
25    <txt-record>api=grpc-web</txt-record>
26  </service>
27
28  <!-- Health check endpoint -->
29  <service>
30    <type>_http._tcp</type>
31    <port>8080</port>
32    <txt-record>path=/actuator/health</txt-record>
33    <txt-record>api=health</txt-record>
34  </service>
35 </service-group>

```

Listing 31: star-gateway/deployment/avahi/star-robot.service

14.3 Envoy Proxy Configuration

```

1 admin:
2   address:
3     socket_address:
4       address: 127.0.0.1
5       port_value: 9901
6
7 static_resources:
8   listeners:
9     - name: grpc_web_listener
10      address:
11        socket_address:

```

```

12     address: 0.0.0.0
13     port_value: 8090
14 filter_chains:
15   - filters:
16     - name: envoy.filters.network.http_connection_manager
17       typed_config:
18         "@type":
19           type.googleapis.com/envoy.extensions.filters.network.http_connection
20         codec_type: AUTO
21         stat_prefix: grpc_web
22
23       route_config:
24         name: local_route
25         virtual_hosts:
26           - name: grpc_services
27             domains: ["*"]
28             routes:
29               - match:
30                 prefix: "/"
31                 route:
32                   cluster: grpc_backend
33                   timeout: 60s
34
35             # CORS configuration
36             cors:
37               allow_origin_string_match:
38                 - prefix: "*"
39               allow_methods: "GET, POST, OPTIONS"
40               allow_headers: >
41                 content-type,
42                 x-grpc-web,
43                 x-user-agent,
44                 grpc-timeout
45               max_age: "1728000"
46               expose_headers: >
47                 grpc-status,
48                 grpc-message
49
50       http_filters:
51         # gRPC-Web to gRPC translation
52         - name: envoy.filters.http.grpc_web
53           typed_config:
54             "@type":
55               type.googleapis.com/envoy.extensions.filters.http.grpc_web.v3
56
57         # CORS handling
58         - name: envoy.filters.http.cors
59           typed_config:
60             "@type":
61               type.googleapis.com/envoy.extensions.filters.http.cors.v3.Cor
62
63         # Router
64         - name: envoy.filters.http.router
65           typed_config:

```

```
63         "@type":
64             type.googleapis.com/envoy.extensions.filters.http.router.v3.Router
65     clusters:
66     - name: grpc_backend
67       connect_timeout: 0.5s
68       type: LOGICAL_DNS
69       dns_lookup_family: V4_ONLY
70       lb_policy: ROUND_ROBIN
71       typed_extension_protocol_options:
72         envoy.extensions.upstreams.http.v3.HttpProtocolOptions:
73           "@type":
74             type.googleapis.com/envoy.extensions.upstreams.http.v3.HttpProtocolOptions
75           explicit_http_config:
76             http2_protocol_options: {}
77       load_assignment:
78         cluster_name: grpc_backend
79         endpoints:
80         - lb_endpoints:
81           - endpoint:
82             address:
83               socket_address:
84                 address: 127.0.0.1
85                 port_value: 9090
```

Listing 32: /opt/envoy/envoy.yaml

15 Implementation Checklist

15.1 Phase 1: Infrastructure Setup

- ☐ Create `star-gateway/` Spring Boot project structure
- ☐ Create `proto/` shared protobuf definitions
- ☐ Set up Gradle build with gRPC + GraphQL plugins
- ☐ Add Envoy to Buildroot configuration
- ☐ Configure development environment (IDE, linting, testing)

15.2 Phase 2: Backend Core

- ☐ Implement `Esp32SpiAdapter` (Pi4J SPI communication)
- ☐ Implement `Esp32TcpAdapter` (TCP fallback)
- ☐ Implement `RplidarAdapter` (SLAMTEC protocol via `rplidar_ros2`)
- ☐ Implement domain services:
 - ☐ `MotorControlService`
 - ☐ `TelemetryService`
 - ☐ `NavigationService`
 - ☐ `SensorFusionService`
- ☐ Implement GraphQL controllers and schema
- ☐ Implement gRPC service for motor control
- ☐ Add Prometheus metrics and health endpoints
- ☐ Write unit tests for all services

15.3 Phase 3: Frontend PWA

- ☐ Copy UI-Demo to `star-ui/`
- ☐ Add PWA dependencies and Vite PWA plugin
- ☐ Create Apollo Client with offline persistence
- ☐ Create gRPC-Web client with proto compilation
- ☐ Replace `useMockData` hook with `useRobotConnection`
- ☐ Implement IndexedDB storage layer
- ☐ Create service worker with Workbox
- ☐ Add offline UI indicators
- ☐ Test PWA installation and offline mode

15.4 Phase 4: ESP32 Updates

- ☐ Create `star_module_tcp` library
- ☐ Implement TCP server with JSON parsing
- ☐ Add SPI peripheral command handler (if not present)
- ☐ Define binary command protocol
- ☐ Integrate TCP server into `main.c`
- ☐ Test SPI + TCP redundancy

15.5 Phase 5: Deployment

- ☐ Create systemd service file
- ☐ Create Avahi mDNS service file
- ☐ Configure Envoy proxy
- ☐ Update Buildroot to include:
 - ☐ OpenJDK 17
 - ☐ Envoy proxy
 - ☐ Gateway JAR
- ☐ Create deployment scripts
- ☐ Integration testing on hardware

15.6 Phase 6: Documentation

- ☐ Update `GEMINI.md` with new architecture
- ☐ Create API documentation (GraphQL schema + gRPC)
- ☐ Update hardware testing guide
- ☐ Create troubleshooting guide
- ☐ Document deployment procedures

16 Technology Summary

Layer	Technology	Purpose
Frontend (PWA)		
UI Framework	Vue 3 + TypeScript + Vite 6	Reactive UI with Composition API
PWA	Workbox 7 + Vite PWA	Offline capability, installability
GraphQL Client	Vue Apollo 4 + Apollo Client	Data fetching, caching, subscriptions
gRPC Client	grpc-web 0.15	Low-latency motor commands
Offline Storage	IndexedDB (idb 8)	Full telemetry history offline
Styling	Tailwind CSS 4	Utility-first responsive design
Backend (Gateway)		
Framework	Spring Boot 3.2	Production-ready JVM framework
Language	Kotlin 1.9	Modern JVM language with coroutines
GraphQL Server	Spring GraphQL	Queries, mutations, subscriptions
gRPC Server	grpc-spring-boot-starter 3.1	Binary RPC for motor control
Hardware Access	Pi4J 2.6+ (GpioD)	SPI/GPIO access on RPi5 RP1 chip
Metrics	Micrometer + Prometheus	Health monitoring, observability
Infrastructure		
Proxy	Envoy 1.29	gRPC-Web to gRPC translation
Service Discovery	Avahi (mDNS)	Zero-config <code>star-robot.local</code>
Process Manager	systemd	Service lifecycle, auto-restart
OS	Buildroot 2025.02 LTS	Minimal Linux for RPi5
Firmware (ESP32)		
Framework	ESP-IDF 5.x	Official Espressif SDK
RTOS	FreeRTOS	Real-time task scheduling
Communication	SPI + TCP/WiFi	Dual-channel redundancy
Architecture	STAR Firmware (DIP)	Modular, testable embedded code

Table 10: Complete Technology Stack

17 References and Resources

17.1 Protocol Documentation

- gRPC Official Documentation: <https://grpc.io/docs/>
- GraphQL Specification: <https://spec.graphql.org/>
- gRPC-Web GitHub: <https://github.com/grpc/grpc-web>
- Spring GraphQL: <https://spring.io/projects/spring-graphql>

17.2 Framework Documentation

- Spring Boot 3: <https://docs.spring.io/spring-boot/docs/3.2.x/reference/>
- Vue Apollo: <https://v4.apollo.vuejs.org/>
- Workbox: <https://developer.chrome.com/docs/workbox/>
- Pi4J: <https://pi4j.com/documentation/>

17.3 Research References

- Hasura: “GraphQL vs gRPC” - <https://hasura.io/learn/graphql/intro-graphql/graphql-vs-grpc/>
- Stack Overflow Blog: “When to use gRPC vs GraphQL” - <https://stackoverflow.blog/2022/11/28/when-to-use-grpc-vs-graphql/>
- Ambassador Labs: “Protobuf vs JSON Performance” - <https://www.getambassador.io/blog/protobuf-vs-json>
- Postman: “gRPC vs GraphQL” - <https://blog.postman.com/grpc-vs-graphql/>

17.4 Build Tools

- Vite: <https://vitejs.dev/>
- Gradle: <https://gradle.org/>
- PlatformIO: <https://platformio.org/>